

# Graph Transformer Networks: Learning meta-path graphs to improve GNNs

Seongjun Yun, Minbyul Jeong, Sungdong Yoo, Seunghun Lee, Sean S. Yi, Raehyun Kim, Jaewoo Kang, Hyunwoo J. Kim\*

Department of Computer Science and Engineering, Korea University, Seoul, South Korea

## ARTICLE INFO

### Article history:

Received 12 October 2021  
Received in revised form 16 April 2022  
Accepted 27 May 2022  
Available online 4 June 2022

### Keywords:

Graph Neural Networks  
Heterogeneous graphs  
Machine learning on graphs  
Network analysis

## ABSTRACT

Graph Neural Networks (GNNs) have been widely applied to various fields due to their powerful representations of graph-structured data. Despite the success of GNNs, most existing GNNs are designed to learn node representations on the *fixed* and *homogeneous* graphs. The limitations especially become problematic when learning representations on a misspecified graph or a *heterogeneous* graph that consists of various types of nodes and edges. To address these limitations, we propose Graph Transformer Networks (GTNs) that are capable of generating new graph structures, which preclude noisy connections and include useful connections (e.g., meta-paths) for tasks, while learning effective node representations on the new graphs in an end-to-end fashion. We further propose enhanced version of GTNs, Fast Graph Transformer Networks (FastGTNs), that improve scalability of graph transformations. Compared to GTNs, FastGTNs are up to  $230\times$  and  $150\times$  faster in inference and training, and use up to  $100\times$  and  $148\times$  less memory while allowing the *identical* graph transformations as GTNs. In addition, we extend graph transformations to the semantic proximity of nodes allowing *non-local* operations beyond meta-paths. Extensive experiments on both homogeneous graphs and heterogeneous graphs show that GTNs and FastGTNs with non-local operations achieve the state-of-the-art performance for node classification tasks. The code is available: [https://github.com/seongjunyun/Graph\\_Transformer\\_Networks](https://github.com/seongjunyun/Graph_Transformer_Networks)

© 2022 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

## 1. Introduction

Graph Neural Networks (GNNs) have become an increasingly popular tool to learn the representations of graph-structured data. They are widely used in a variety of tasks such as node classification (Kipf & Welling, 2017; Li, Han & Wu, 2018; Liu et al., 2019; Liu, Gao, & Ji, 2020; Xu et al., 2018), link prediction (Kipf & Welling, 2016b; Schlichtkrull et al., 2018; Teru, Denis, & Hamilton, 2020; Vashishth, Sanyal, Nitin, & Talukdar, 2020; Zhang & Chen, 2018), graph classification (Gao & Ji, 2019; Ma, Wang, Aggarwal, & Tang, 2019; Xu, Hu, Leskovec, & Jegelka, 2019; Ying et al., 2018), and graph generation (Dai, Nazi, Li, Dai, & Schuurmans, 2020; Li, Vinyals, Dyer, Pascanu & Battaglia, 2018; Liao et al., 2019; You, Liu, Ying, Pande & Leskovec, 2018; You, Ying, Ren, Hamilton & Leskovec, 2018).

Despite their effectiveness to learn representations on graphs, most GNNs assume that the given graphs are fixed and homogeneous. Since the graph convolutions discussed above are

determined by a fixed graph structure, a noisy graph with missing/spurious connections results in ineffective convolution with wrong neighbors on the graph. In addition, in some applications constructing a graph to operate GNNs is not trivial. For example, a citation network has multiple types of nodes (e.g., authors, papers, conferences) and edges defined by their relations (e.g., author-paper, paper-conference), which are referred to as *heterogeneous* graphs. In heterogeneous graphs, the importance of each node type and edge type can vary depending on the task, and some node/edge types may even become completely useless. A naïve approach to deal with the heterogeneous graphs is to ignore the node/edge types and treat them as in a *homogeneous* graph (a standard graph with one type of nodes and edges). This, apparently, is suboptimal since models cannot exploit the type information. A more recent remedy is to manually design useful meta-paths, which are paths connected with heterogeneous edge types, and transform a heterogeneous graph into a *homogeneous* graph defined by the meta-paths. Then conventional GNNs can operate on the transformed homogeneous graphs (Wang, Ji, et al., 2019; Zhang et al., 2018). Despite improving on previous approaches, this is a two-stage approach and requires hand-crafted meta-paths for each problem. The accuracy

\* Corresponding author.

E-mail addresses: [ysj5419@korea.ac.kr](mailto:ysj5419@korea.ac.kr) (S. Yun), [minbyuljeong@korea.ac.kr](mailto:minbyuljeong@korea.ac.kr) (M. Jeong), [ysd424@korea.ac.kr](mailto:ysd424@korea.ac.kr) (S. Yoo), [llsshh319@korea.ac.kr](mailto:llsshh319@korea.ac.kr) (S. Lee), [seanswyi@korea.ac.kr](mailto:seanswyi@korea.ac.kr) (S.S. Yi), [raehyun@korea.ac.kr](mailto:raehyun@korea.ac.kr) (R. Kim), [kangji@korea.ac.kr](mailto:kangji@korea.ac.kr) (J. Kang), [hyunwoojkim@korea.ac.kr](mailto:hyunwoojkim@korea.ac.kr) (H.J. Kim).

of downstream analysis can be significantly affected by the choice of these meta-paths.

To address this limitation, we develop the Graph Transformer Networks (GTNs) that learn to transform an original graph into new graphs that preclude noisy connections and include useful multi-hop connections (e.g., meta-paths) for each task, and learn node representation on the new graphs in an end-to-end fashion. Specifically, the Graph Transformer layer, a core layer of GTN, learns a soft selection of adjacency matrices for edge types and multiply two selected adjacency matrices to generate useful meta-paths. Also, by leveraging an identity matrix, GTN can generate new graph structures based on arbitrary-length composite relations connected with softly chosen edge types in a heterogeneous graph.

Furthermore, we address the scalability issue of GTNs. To transform graphs, GTNs *explicitly* compute a new adjacency matrix of meta-paths by the matrix multiplications of huge adjacency matrices. This requires substantial computational costs and large memory making it infeasible to apply GTNs to a large graph. To address this issue, we propose an enhanced version of GTNs, Fast Graph Transformer Networks (FastGTNs), that *implicitly* transform the graphs without the multiplication of two adjacency matrices. Compared to GTNs, FastGTNs are up to  $230\times$  and  $150\times$  faster in inference and training, and use up to  $100\times$  and  $148\times$  less memory while allowing the *identical* graph transformations as GTNs.

Another issue of GTNs is its edge generation is limited to the nodes connected by a meta-path of the input graphs, which do not take into account the semantic proximity of nodes. We further extend graph transformations to the semantic proximity of nodes allowing *non-local* operations beyond meta-paths.

The new graph structures from GTNs and FastGTNs lead to effective node representations resulting in state-of-the-art performance, without any predefined meta-paths from domain knowledge, on six heterogeneous graph datasets. In addition, since GTNs and FastGTNs learn variable lengths of useful meta-paths (i.e., the neighborhood range of each node), GTNs and FastGTNs achieve state-of-the-art performance on six homogeneous graph datasets by adjusting neighborhood ranges for each dataset.

Our **contributions** are as follows:

- (i) We propose a novel framework Graph Transformer Networks, to learn a new graph structure which involves identifying useful meta-paths and multi-hop connections for learning effective node representation on graphs.
- (ii) We propose Fast Graph Transformer Networks (FastGTNs) that *implicitly* transform the graphs without the multiplication of adjacency matrices which requires excessive resources while allowing the identical transformations of GTNs.
- (iii) We extend graph transformations to non-local operations incorporating the node features to utilize the semantic proximity of nodes beyond meta-paths.
- (iv) We prove the effectiveness of node representation learnt by Graph Transformer Networks and FastGTNs with non-local operations resulting in the best performance against state-of-the-art GNN-baselines in six benchmark node classification on heterogeneous graphs and six benchmark node classification on homogeneous graphs. Also, our experiments demonstrate that on a large graph dataset FastGTNs show  $230\times$  faster inference time and  $100\times$  less memory usage than the GTNs.

This work builds upon the preliminary conference paper (Yun, Jeong, Kim, Kang, & Kim, 2019) with the following substantial improvements. (1) We propose an enhanced version of Graph

Transformer Networks, Fast Graph Transformer Networks (FastGTNs), that improve the scalability of graph transformations, which are  $230\times$  faster and use  $100\times$  less memory in inference while allowing identical graph transformations as GTNs. (2) We extend graph transformations to non-local operations incorporating the node features to utilize the semantic proximity of nodes beyond meta-paths. (3) We provide relationships between our FastGTNs and other popular GNN architectures, which show that several popular graph neural networks such as GCN and MixHop can be special cases of our FastGTNs. (4) We show the effectiveness of both GTNs and FastGTNs with non-local operations on additional three large-scale heterogeneous graph datasets and six homogeneous graph datasets. (5) We compared our methods to recent baselines on both homogeneous and heterogeneous graph datasets.

## 2. Related works

Recent years have witnessed significant development in deep learning architectures for graphs. Bruna, Zaremba, Szlam, and Lecun (2014) first proposed a convolution operation on graphs leveraging the Fourier transform and convolution kernels in a spectral domain. Defferrard, Bresson, and Vandergheynst (2016), Li, Han and Wu (2018) extended and improved spectral-based Graph Convolutional Networks (GCNs). On the other hand, spatial-based GNNs (Abu-El-Haija et al., 2019; Hamilton, Ying, & Leskovec, 2017; Veličković et al., 2018; Xu et al., 2019, 2018) have been proposed to improve the scalability of GNNs by performing graph convolution operations in the graph domain. Kipf and Welling (2017) proposed the first-order approximation of the spectral filter using the Chebyshev polynomials. GCNs, the simplified spectral-based GNNs, can be viewed as spatial-based GNNs as well. Graph Attention Networks (GATs) (Veličković et al., 2018) incorporate the attention mechanism into GNNs, which differentially aggregate the representations of neighbors on graphs using attention scores. GraphSAGE (Hamilton et al., 2017) expanded the operating range of GNNs for the inductive setting generating representations for unseen nodes by variable aggregation operations. Jumping Knowledge Networks (JKNets) (Xu et al., 2018) utilized flexible neighborhood ranges by adopting the skip-connection and Mixhop (Abu-El-Haija et al., 2019) leveraged a combination of powers of normalized adjacency matrices to aggregate features at various distances.

In contrast to the above works on *homogeneous* graphs, several studies (Hu, Dong, Wang, & Sun, 2020; Schlichtkrull et al., 2018; Wang, Ji, et al., 2019) have attempted to extend GNN architectures to *heterogeneous* graphs that contain multiple types of nodes and edges. They are categorized into two approaches: GNNs with relation-specific parameters (Hu et al., 2020; Schlichtkrull et al., 2018) and GNNs with relation-based graph transformations (Wang, Ji, et al., 2019). Relational Graph Convolutional Networks (R-GCNs) (Schlichtkrull et al., 2018) employed GCNs with relation-specific convolutions (or weight matrices) to deal with heterogeneous graphs. Hu et al. (2020) proposed the Heterogeneous Graph Transformer (HGT) to parameterize the meta relation triplet of each edge type and used a structure that utilizes the self-attention of the transformer architecture (Vaswani et al., 2017) to learn specific patterns of different relationships. The second approaches, GNNs with relation-based graph transformations, generally utilize meta-paths. The Heterogeneous Graph Attention Network (HAN) (Wang, Ji, et al., 2019) first transforms heterogeneous graphs into homogeneous graphs using manually selected meta-paths and applies an attention-based GNN on the graphs. However, the HAN has limitations that it is a multi-stage approach and requires the manual selection of meta-paths in each dataset. Also, performance can be significantly affected

by the choice of meta-paths. Unlike this approach, our Graph Transformer Networks can operate on a heterogeneous graph and transform the graph for tasks while learning node representations on the transformed graphs in an end-to-end fashion.

### 3. Method

The goal of our framework, Graph Transformer Networks, is to generate new graph structures and learn node representations on the learned graphs simultaneously. Unlike most GNNs on graphs that assume the graph is given, GTNs seek for new graph structures using multiple candidate adjacency matrices to perform more effective graph convolutions and learn more powerful node representations. Learning new graph structures involves identifying useful meta-paths, which are paths connected with heterogeneous edges, and multi-hop connections. Before introducing our framework, we first briefly review the basic notions of heterogeneous graphs and GCN, and then introduce our Graph Transformer Networks.

#### 3.1. Preliminaries

**Heterogeneous Graph** (Shi, Li, Zhang, Sun, & Philip, 2016). Let  $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_v, \mathcal{T}_e)$  denote a directed graph where each node  $v \in \mathcal{V}$  and each edge  $e \in \mathcal{E}$  are associated with their type mapping functions  $\tau_v(v) : \mathcal{V} \rightarrow \mathcal{T}_v$  and  $\tau_e(e) : \mathcal{E} \rightarrow \mathcal{T}_e$ , respectively. The heterogeneous graph  $\mathcal{G}$  can be represented by a set of adjacency matrices  $\{A_t\}_{t=1}^{|\mathcal{T}_e|}$  or a tensor (i.e.,  $\mathbb{A} \in \mathbf{R}^{|\mathcal{V}| \times |\mathcal{V}| \times |\mathcal{T}_e|}$ ), where  $A_t \in \mathbf{R}^{N \times N}$  is an adjacency matrix of the  $t$ th edge type and  $|\mathcal{V}| = N$ .  $A_t[i, j]$  denotes the weight of the  $t$ th type edge from node  $j$  to node  $i$ . When a graph has a single type of nodes and edges, i.e.,  $|\mathcal{T}_v| = 1$  and  $|\mathcal{T}_e| = 1$ , it is called a *homogeneous* graph.

**Meta-Path** (Wang, Ji, et al., 2019). In heterogeneous graphs, a multi-hop connection is called a *meta-path*, which is a path connected with heterogeneous edge types, i.e.,  $v_1 \xrightarrow{\tau_e(e_1)} v_2 \xrightarrow{\tau_e(e_2)} \dots \xrightarrow{\tau_e(e_\ell)} v_{\ell+1}$ , where  $\tau_e(e_\ell) \in \mathcal{T}_e$  denotes the edge type of edge  $e_\ell$  on the meta-path.

**Graph Convolutional network (GCN)**. In this work, a graph convolutional network (GCN) (Kipf & Welling, 2016a) is used to learn useful representations for node classification in an end-to-end fashion. Let  $H^{(l)}$  be the feature representations of the  $l$ th layer in GCNs, the forward propagation becomes

$$H^{(l+1)} = \sigma \left( \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} \right), \quad (1)$$

where  $\tilde{A} = A + I \in \mathbf{R}^{N \times N}$  is the adjacency matrix  $A$  of the graph  $\mathcal{G}$  with added self-connections,  $\tilde{D}$  is the degree matrix of  $\tilde{A}$ , i.e.,  $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$ , and  $W^{(l)} \in \mathbf{R}^{d \times d}$  is a trainable weight matrix. One can easily observe that the convolution operation across the graph is determined by the given graph structure and it is not learnable except for the node-wise linear transform  $H^{(l)} W^{(l)}$ . So the convolution layer can be interpreted as the composition of a fixed convolution followed by an activation function  $\sigma$  on the graph after a node-wise linear transformation. Since we learn graph structures, our framework benefits from the different convolutions, namely  $\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}}$ , obtained from learned multiple adjacency matrices. The architecture will be introduced later in this section. For a directed graph (i.e., asymmetric adjacency matrix),  $\tilde{A}$  in (1) can be normalized by the inverse of in-degree diagonal matrix  $D^{-1}$  as  $H^{(l+1)} = \sigma(\tilde{D}^{-1} \tilde{A} H^{(l)} W^{(l)})$ .

#### 3.2. Learning meta-path graphs

Previous works (Wang, Ji, et al., 2019; Zhang et al., 2018) require manually defined meta-paths and perform Graph Neural Networks on the meta-path graphs. Instead, our Graph Transformer Networks (GTNs) learn meta-path graphs for given data and tasks and operate graph convolution on the learned meta-path graphs. This gives a chance to find more useful meta-paths and lead to virtually various graph convolutions using multiple meta-path graphs.

The main idea of learning meta-path graphs is that a new adjacency matrix  $A_{\mathcal{P}}$  of a useful meta-path  $\mathcal{P}$  connected by a particular order of edge types (e.g.,  $t_1, t_2 \dots t_\ell$ ) is obtained by the multiplications of adjacency matrices of the edge types as

$$A_{\mathcal{P}} = A_{t_\ell} \dots A_{t_2} A_{t_1}. \quad (2)$$

Based on this idea, each  $k$ th Graph Transformer (GT) Layer in GTN learns to softly select adjacency matrices (edge types) by  $1 \times 1$  convolution with the weights from softmax function as

$$F(\mathbb{A}; \phi^{(k)}) = \text{conv}_{1 \times 1}(\mathbb{A}; \text{softmax}(\phi^{(k)})) \quad (3)$$

$$= \sum_{t=1}^{|\mathcal{T}_e|} \alpha_t^{(k)} A_t, \quad (4)$$

where  $\phi^{(k)} \in \mathbf{R}^{1 \times 1 \times |\mathcal{T}_e|}$  is the parameter of  $1 \times 1$  convolution and  $\alpha^{(k)} = \text{softmax}(\phi^{(k)})$  (i.e., the convex combination of adjacency matrices as  $\alpha^{(k)} \cdot \mathbb{A}$ ). Then the meta-path adjacency matrix is computed by matrix multiplication of an output and the output matrix of the previous  $(k-1)$ -th GT Layer as  $A^{(k-1)} F(\mathbb{A}; \phi^{(k)})$ . For numerical stability, the matrix is normalized by its degree matrix as

$$A^{(k)} = \left( \hat{D}^{(k)} \right)^{-1} A^{(k-1)} F(\mathbb{A}; \phi^{(k)}), \quad (5)$$

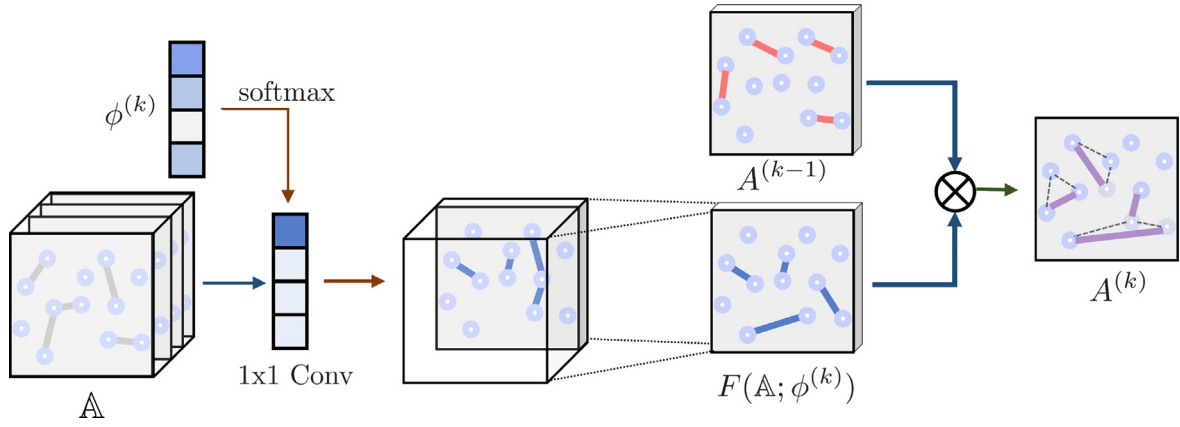
where  $A^{(0)} = F(\mathbb{A}; \phi^{(0)})$  and  $\hat{D}^{(k)}$  is a degree matrix of the output after the multiplication of two matrices  $A^{(k-1)} F(\mathbb{A}; \phi^{(k)})$ . Now, we need to check whether GTN can learn an arbitrary meta-path with respect to edge types and path length. The adjacency matrix of arbitrary length  $k+1$  meta-paths can be calculated by

$$A_{\mathcal{P}} = \left( \sum_{t_0 \in \mathcal{T}_e} \alpha_{t_0}^{(0)} A_{t_0} \right) \left( \sum_{t_1 \in \mathcal{T}_e} \alpha_{t_1}^{(1)} A_{t_1} \right) \dots \left( \sum_{t_k \in \mathcal{T}_e} \alpha_{t_k}^{(k)} A_{t_k} \right) \quad (6)$$

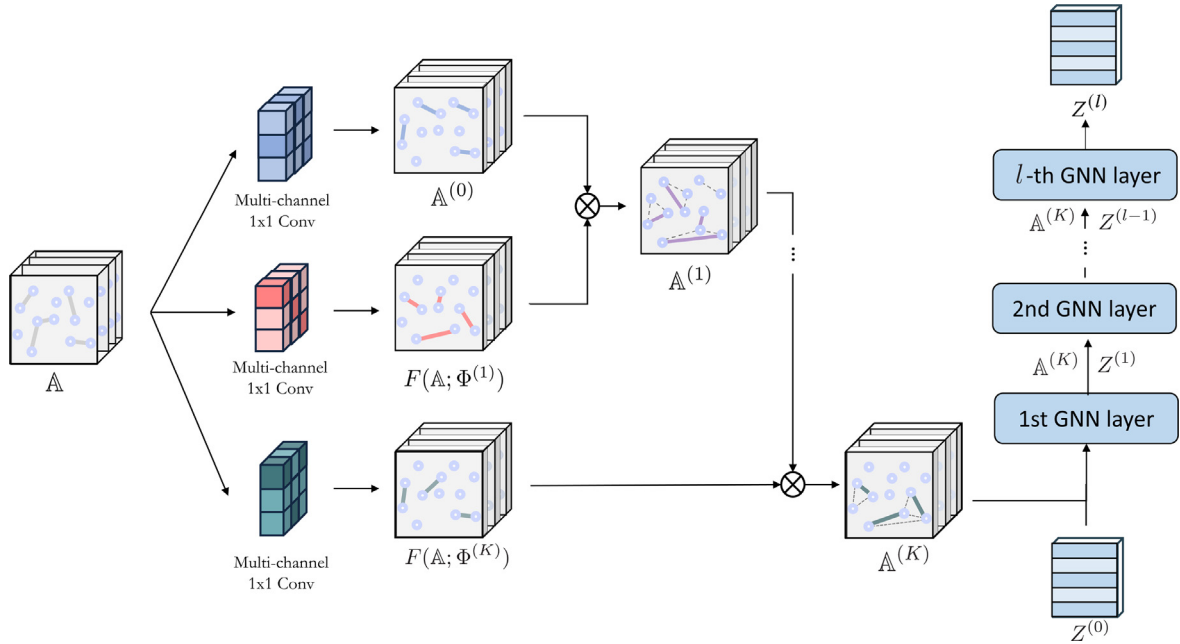
where  $A_{\mathcal{P}}$  denotes the adjacency matrix of meta-paths,  $\mathcal{T}_e$  denotes a set of edge types and  $\alpha_{t_k}^{(k)}$  is the weight for edge type  $t_k$  at the  $k$ th GT layer. When  $\alpha$  is not one-hot vector,  $A_{\mathcal{P}}$  can be seen as the weighted sum of all length- $(k+1)$  meta-path adjacency matrices. So a stack of  $k$  GT layers allows to learn length  $k+1$  meta-path graph structures as the architecture of GTN shown in Fig. 2. One issue with this construction is that adding GT layers always increase the length of meta-path and this does not allow the original edges. In some applications, both long meta-paths and short meta-paths are important. To learn short and long meta-paths including original edges, we include the identity matrix  $I$  in  $\mathbb{A}$ , i.e.,  $A_0 = I$ . This trick allows GTNs to learn any length of meta-paths up to  $k+1$  when  $k$  GT layers are stacked.

#### 3.3. Graph transformer networks

We here introduce the architecture of Graph Transformer Networks. To consider multiple types of meta-paths simultaneously, the GTN generates multiple graph structures by setting the output channels of  $1 \times 1$  filter to  $C$ . Then the output matrix of  $k$ th GT layer  $A^{(k)}$  becomes the output tensor  $\mathbb{A}^{(k)} \in \mathbf{R}^{N \times N \times C}$  and the weight vector  $\phi^{(k)}$  of the  $k$ th GT Layer becomes the weight matrix



**Fig. 1. Graph Transformer Layer** softly selects adjacency matrices (edge types) from the set of adjacency matrices  $\mathbb{A}$  of a heterogeneous graph  $G$  and learns a new meta-path graph represented by  $\mathbb{A}^{(k)}$  via the matrix multiplication of the output matrix of the previous  $(k - 1)$ th GT Layer and the selected adjacency matrix  $F(\mathbb{A}; \phi^{(k)})$ . The soft adjacency matrix selection is a weighted sum of candidate adjacency matrices obtained by  $1 \times 1$  convolution with non-negative weights from  $\text{softmax}(\phi^{(k)})$ .



**Fig. 2. Graph Transformer Networks (CTNs)** learn to generate a set of new meta-path adjacency matrices  $\mathbb{A}^{(K)}$  using GT layers and perform graph convolution as in GCNs on the new graph structures. Multiple node representations from the same GCNs on multiple meta-path graphs are integrated by concatenation and improve the performance of node classification.  $F(\mathbb{A}; \Phi^{(K)})$  is an intermediate adjacency tensor to compute meta-paths at the  $K$ th layer.

$\Phi^{(k)}$ , Eq. (5), can be represented in the form of tensor equations as

$$\mathbb{A}^{(k)} = (\hat{\mathbb{D}}^{(k)})^{-1} \mathbb{A}^{(k-1)} * F(\mathbb{A}; \Phi^{(k)}), \quad (7)$$

where  $\mathbb{A}^{(k-1)} * F(\mathbb{A}; \Phi^{(k)}) = \big\|_{c=1}^C A_c^{(k-1)} F(\mathbb{A}; \phi^{(k,c)})$  and  $\hat{\mathbb{D}}^{(k)}$  is a degree tensor of the output after the multiplication of two tensors  $\mathbb{A}^{(k-1)} * F(\mathbb{A}; \Phi^{(k)})$ . It is beneficial to learn different node representations via multiple different graph structures.

After the stack of  $K$  GT Layers, multi-layer GNNs are applied to the each channel of the output tensor  $\mathbb{A}^{(K)}$  and update node representations  $Z$  as follows:

$$Z^{(l+1)} = f_{\text{agg}} \left( \big\|_{c=1}^C \sigma(\tilde{D}_c^{-1} \tilde{A}_c^{(K)} Z^{(l)} W^{(l)}) \right), \quad (8)$$

where  $\|$  is the concatenation operator,  $C$  denotes the number of channels,  $Z^{(l)}$  denotes the node representations at the  $l$ th GNN layer,  $\tilde{A}_c^{(K)} = A_c^{(K)} + \gamma I$  is the adjacency matrix with self-loops from the  $c$ th channel of  $\mathbb{A}^{(K)}$ ,  $\tilde{D}_c$  is the degree matrix of  $\tilde{A}_c^{(K)}$ ,  $W^{(l)} \in \mathbf{R}^{d^{(l)} \times d^{(l+1)}}$  is a trainable weight matrix shared across channels,  $Z^{(0)}$  is a feature matrix  $X \in \mathbf{R}^{N \times F}$  and  $f_{\text{agg}}$  is a channel aggregation function.

The final node representations  $Z^{(l)}$  after  $l$  GNN layers can then be used for downstream tasks. For the node classification task, we applied dense layers followed by a softmax layer to the node representations. Then with ground truth labels of nodes, we can optimize the model weights by minimizing the cross-entropy via backpropagation and gradient descent.

### 3.4. Fast graph transformer networks

In the previous sections, we demonstrated that GTNs can transform the original graphs into new meta-path graphs while



learning representations on the meta-path graphs. However, GTNs have a scalability issue. GTNs *explicitly* compute a new adjacency matrix of meta-paths by the matrix multiplication of two adjacency matrices and store the new adjacency matrix at each layer. So a graph transformation in GTNs involves huge computational costs and large memory. This makes it infeasible to apply GTNs to a large graph. To address these issues, we develop the enhanced version of GTNs, FastGTNs, which *implicitly* transform the graph structures without storing the new adjacency matrices of meta-paths. In this section, we describe our FastGTNs in detail.

To derive FastGTNs, we first begin with the equation of GTNs. Our goal here is to derive a new architecture without the need for the explicit multiplications of large adjacency matrices. For simplicity, we assume that the number of channels is one, i.e.,  $C=1$ , one GCN Layer is applied on top of the new graph structure and a channel aggregation function is an identity function i.e.,  $f_{agg}(x) = x$ . Then the node representations  $Z$  of the GTNs are given as follows:

$$Z = \sigma(\tilde{D}^{-1}(A^{(K)} + I)XW), \quad (9)$$

where  $A^{(K)} \in \mathbf{R}^{N \times N}$  is a new adjacency matrix from a GTN with  $K$  GT layers, and  $X \in \mathbf{R}^{N \times F}$  is input features,  $W \in \mathbf{R}^{F \times d'}$  is a linear transformation in a GCN layer,  $\tilde{D}^{-1} \in \mathbf{R}^{N \times N}$  is an inverse degree matrix of  $(A^{(K)} + I)$ . We observe that a GT Layer multiplies two softly selected adjacency matrices and normalizes the output adjacency matrix. So (9) can be written as

$$\begin{aligned} Z &= \sigma(\tilde{D}^{-1}XW + \tilde{D}^{-1}A^{(K)}XW) \\ &= \sigma(\tilde{D}^{-1}XW + \tilde{D}^{-1} \left( (\hat{D}^{(K)})^{-1} A^{(K-1)}(\alpha^{(K)} \cdot \mathbb{A}) \right) XW) \\ &= \sigma(\tilde{D}^{-1}XW + \tilde{D}^{-1} \left( (\hat{D}^{(K)})^{-1} \dots \left( (\hat{D}^{(1)})^{-1} (\alpha^{(0)} \cdot \mathbb{A})(\alpha^{(1)} \cdot \mathbb{A}) \right) \dots (\alpha^{(K)} \cdot \mathbb{A}) \right) XW). \end{aligned} \quad (10)$$

Eq. (10) clearly shows the computational bottleneck in (10) is the multiplications of huge adjacency matrices, e.g.,  $(\alpha^{(0)} \cdot \mathbb{A})(\alpha^{(1)} \cdot \mathbb{A}) \dots (\alpha^{(K)} \cdot \mathbb{A})$ . To resolve this problem, we can rewrite (10) using the associative property of matrix multiplication as

$$Z = \sigma(\tilde{D}^{-1}XW + \tilde{D}^{-1} \left( \hat{D}^{(K)} \right)^{-1} \dots \left( \hat{D}^{(1)} \right)^{-1} (\alpha^{(0)} \cdot \mathbb{A})(\alpha^{(1)} \cdot \mathbb{A}) \dots (\alpha^{(K)} \cdot \mathbb{A})XW). \quad (11)$$

Now, Eq. (11) implies that at each layer, without the *matrix multiplications* of huge adjacency matrices, the identical features can be obtained by a sequence of *feature* transformations using a differently constructed adjacency matrix. It efficiently reduces the computational cost from  $O(N^3)$  to  $O(N^2F)$  and memory usage from  $O(N^2)$  to  $O(NF)$ . However, note that since we do not compute the multiplication of two adjacency matrices anymore, we cannot compute degree matrices  $\tilde{D}^{-1} \left( \hat{D}^{(K)} \right)^{-1} \dots \left( \hat{D}^{(1)} \right)^{-1}$ .

To address this challenge, now we show that given a condition of input data and a proposition of normalized matrices, we can make all degree matrices in (11) into identity matrices. Then we can compute (11) without the matrix multiplications of huge adjacency matrices. We begin with a proposition and a condition, then derive an equation of our FastGTNs from (11) by using Proposition 1 (see Fig. 3).

**Proposition 1.** Given two normalized adjacency matrices  $A, B \in \mathbf{R}^{N \times N}$ , the followings are equivalent:

$$(i) (D_A^{-1}A)(D_B^{-1}B) = (D_{AB}^{-1}AB)$$

$$\begin{aligned} (ii) D_{AB}^{-1} &= I \\ (iii) D_{A+I}^{-1} &= (D_A + I)^{-1} = \frac{1}{2}I \end{aligned}$$

The proof of Proposition 1 is provided in A.2 in the supplement. We first assume that each adjacency matrix in  $\mathbb{A}$  is row-wise normalized i.e.,  $\sum_j A_t[i, j] = 1$ . The convex combination of adjacency matrices at each  $k$ th layer i.e.,  $\alpha^{(k)} \cdot \mathbb{A}$  is also a normalized matrix. Then  $(\hat{D}^{(k)})^{-1}$  is an inverse degree matrix of the output after multiplication of two normalized matrices  $A^{(k-1)}(\alpha^{(K)} \cdot \mathbb{A})$ . It means that by (ii) in Proposition 1, all  $(\hat{D}^{(k)})^{-1}$  at each  $k$ th layer are the identity matrix  $I$  and, thus, (11) can be rewritten as

$$Z = \sigma(\tilde{D}^{-1}XW + \tilde{D}^{-1}(\alpha^{(0)} \cdot \mathbb{A}(\alpha^{(1)} \cdot \mathbb{A} \dots (\alpha^{(K)} \cdot \mathbb{A})XW))). \quad (12)$$

By (iii) in Proposition 1, we can also know that  $\tilde{D}^{-1} = (D_{A^{(K)}} + I)^{-1} = \frac{1}{2}I$ , then  $Z$  can be represented as

$$Z = \sigma\left(\frac{1}{2}XW + \frac{1}{2}(\alpha^{(0)} \cdot \mathbb{A}(\alpha^{(1)} \cdot \mathbb{A} \dots (\alpha^{(K)} \cdot \mathbb{A})XW))\right). \quad (13)$$

Now this derivation means that our FastGTNs are not an approximation of GTNs. Mathematically, they are exactly identical. We will also empirically show the identity in Section 4.3. We reverse the order of layers from 0 to  $K$  and replace  $\frac{1}{2}$  with a hyper-parameter  $\gamma$ , then the output of a FastGTN can be represented as

$$Z = \sigma(\gamma XW + (1 - \gamma)(\alpha^{(K)} \cdot \mathbb{A} \dots (\alpha^{(0)} \cdot \mathbb{A})XW)). \quad (14)$$

We finally rewrite our FastGTNs for multi-channel and multi-layer settings as

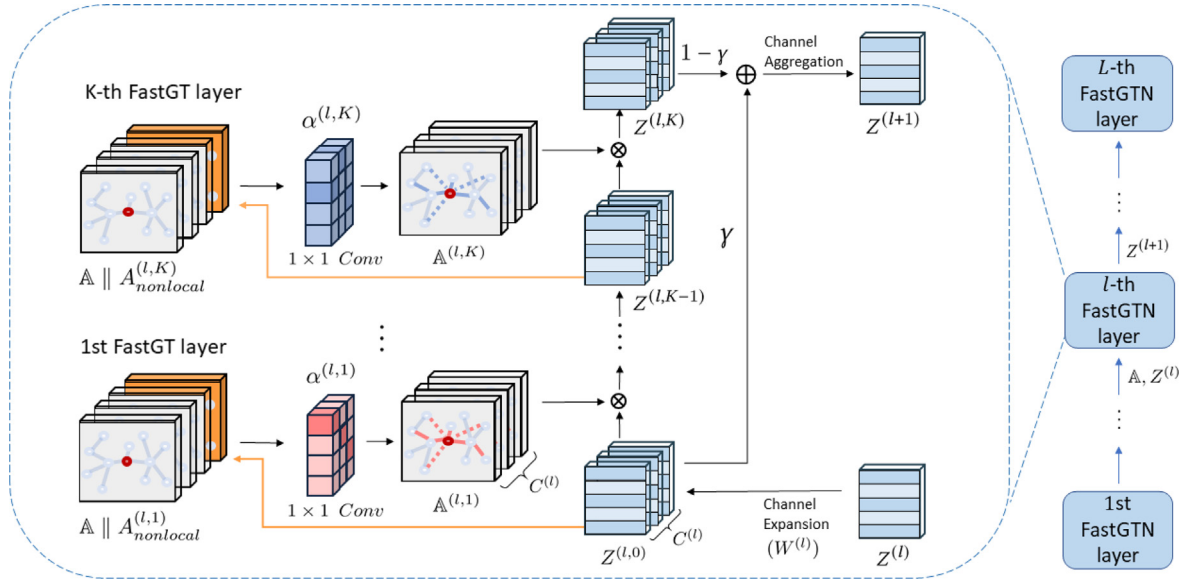
$$Z^{(l+1)} = f_{agg} \left( \bigg\|_{c=1}^{C^{(l)}} \sigma(\gamma Z^{(l)} W_c^{(l)} + (1 - \gamma) Z_c^{(l,K)}) \right), \quad (15)$$

$$Z_c^{(l,K)} = (\alpha^{(l,K,c)} \cdot \mathbb{A} \dots (\alpha^{(l,0,c)} \cdot \mathbb{A}) Z_c^{(l)} W_c^{(l)}), \quad (16)$$

where  $C^{(l)}$  denotes the number of channels,  $Z^{(l)}$  denotes the node representations from the  $l$ th FastGTN layer,  $W_c^{(l)} \in \mathbf{R}^{d^{(l)} \times d^{(l+1)}}$  is a linear transformation in  $c$ th channel of the  $l$ th FastGTN layer,  $\mathbb{A}$  is a set of normalized adjacency matrices,  $\alpha^{(l,k,c)}$  is a convolution filter in the  $c$ th channel of the  $k$ th FastGT layer in the  $l$ th FastGTN layer,  $Z^{(0)}$  is a feature matrix  $X \in \mathbf{R}^{N \times F}$  and  $f_{agg}$  is a channel aggregation function. Furthermore, to deal with huge graphs with about 30 million edges, we additionally propose a mini-batch training algorithm for GTNs and FastGTNs in A.1 in the supplement.

### 3.5. Non-local operations.

One limitation of the GTNs is that its transformation is limited to compositions of existing relations. Specifically,  $K$  GT layers can generate edges only up to  $(K+1)$ -hop relations. It cannot generate remote relations based on semantic proximity between nodes. To address this limitation, we extend graph transformations to non-local operations incorporating the node features to utilize the semantic proximity of nodes beyond meta-paths. However, as mentioned in the previous section, since GTN itself requires large computation cost, we extend non-local operations only to FastGTNs. Specifically, at each  $k$ th FastGT layer in each  $l$ th FastGTN layer, we construct a non-local adjacency matrix  $A_{\text{non-local}}^{(l,k)} \in \mathbf{R}^{N \times N}$  based on hidden representations  $Z^{(l,k-1)}$  from the previous FastGT layer and append the non-local adjacency matrix to the candidate set of adjacency matrices  $\mathbb{A}$  to utilize the non-local relations for graph transformations. To construct  $A_{\text{non-local}}^{(l,k)}$ , we first calculate a



**Fig. 3.** Fast Graph Transformer Networks (FastGTNs) implicitly transform graph structures by a sequence of feature transformations using differently constructed adjacency matrices from Fast Graph Transformer Layers (FastGT Layers). Each  $k$ th FastGT Layer first generates a non-local adjacency matrix  $A_{nonlocal}^{(l,k)}$  using the hidden representations  $Z^{(l,k-1)}$  from  $(k-1)$ th FastGT layer and appends it to the set of adjacency matrices  $\mathbb{A}$ . To generate diverse graphs,  $k$ th FastGT layer generates  $C^{(l)}$  new adjacency matrices  $\mathbb{A}^{(l,k)} \in \mathbb{R}^{N \times N \times C^{(l)}}$  by applying a  $1 \times 1$  convolution filter. Using the  $C^{(l)}$  new adjacency matrices in  $\mathbb{A}^{(l,k)}$ , a FastGT layer transforms the hidden presentations  $Z^{(l,k-1)}$  into  $Z^{(l,k)}$ . After  $K$  FastGT layers, the final representations are obtained by channel aggregation after a convex combination of the expanded input representations  $Z^{(l,0)}$  and the output representations of the  $K$ th FastGT layer, i.e.,  $Z^{(l+1)} = f_{agg}(\gamma Z^{(l,0)} + (1-\gamma)Z^{(l,K)})$ , where  $\gamma \in (0, 1)$ .

graph affinity matrix at each  $k$ th FastGT Layer  $M^{(l,k)}$  based on the similarity between node features. We take an average of multi-channel hidden representations from each  $(k-1)$ th FastGT Layer and project the averaged representations into a latent space by a non-linear transformation  $g_\theta$ . Then we compute the affinity matrix  $M^{(l,k)}$  using the similarity in the latent space as

$$M^{(l,k)} = (g_\theta(H^{(l,k-1)}))g_\theta(H^{(l,k-1)})^T, \quad (17)$$

$$H^{(l,k-1)} = \frac{1}{C^{(l)}} \sum_{c=1}^{C^{(l)}} Z_c^{(l,k-1)}, \quad (18)$$

where  $Z^{(l,k-1)}$  denotes hidden representations at a  $(k-1)$ th FastGT Layer. We use the trick of a decoder in GAE (Kipf & Welling, 2016b) as the similarity function to get the affinity matrix. The affinity matrix can be seen as a weighted adjacency matrix of the fully connected graph. If we include the dense affinity matrix as an adjacency matrix for graph transformation, it causes huge computation cost and may rather propagate irrelevant information between nodes. Therefore, we sparsify the affinity matrix by extracting only  $n$  largest weights for each node  $i$  and construct non-local adjacency matrix  $A_{non\ local}^{(l,k)}$  as

$$A_{non\ local}^{(l,k)}[i, j] = \begin{cases} M_{ij}^{(l,k)}, & \text{if } j \in \arg \text{top } k(M^{(l,k)}[i, :], n) \\ 0, & \text{otherwise} \end{cases} \quad (19)$$

We row-wise normalize the non-local adjacency matrix by applying the softmax function to edge weights of each row. Then the final non-local adjacency matrix at each  $k$ th FastGT layer is represented as

$$A_{non\ local}^{(l,k)}[i, :] = \text{softmax}(\text{topk}(M^{(l,k)}[i, :], n)). \quad (20)$$

To use the normalized non-local adjacency matrix for transformations, we add a non-local parameter to  $1 \times 1$  convolution filters of each FastGT Layer and then append the matrix  $A_{non\ local}^{(l,k)}$  to the set of adjacency matrix of  $k$ th FastGT Layer i.e.,  $\mathbb{A} \parallel A_{non\ local}^{(l,k)}$ . Combining with non-local operations, the overall architecture of FastGTN is presented in Fig. 3.

### 3.6. Relations to other GNN architectures

FastGTNs enhance the scalability of GTNs by implicitly transforming graph structures. Moreover, the FastGTNs become a flexible/general model that subsumes other graph neural networks. In this section, we discuss relationships between our FastGTNs and other GNN architectures. Interestingly, if input graphs are normalized i.e.,  $D = I$ , several popular graph neural networks such as GCN (Kipf & Welling, 2017) and MixHop (Abu-El-Haija et al., 2019) can be special cases of our FastGTNs. In addition, RGCN (Schlichtkrull et al., 2018) can be subsumed by our FastGTNs with minor modifications. We first discuss the graph convolution network (GCN). The GCN computes the output node representations from the  $l$ th GCN layer as

$$Z^{(l+1)} = \sigma(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} Z^{(l)} W^{(l)}) \quad (21)$$

$$= \sigma\left(\frac{1}{2} Z^{(l)} W^{(l)} + \frac{1}{2} A Z^{(l)} W^{(l)}\right), \quad (22)$$

where  $\tilde{A} = A + I \in \mathbb{R}^{N \times N}$  and  $\tilde{D}$  is the degree matrix of  $\tilde{A}$ . If the number of FastGT layers in our FastGTNs is one i.e.,  $K = 1$ , the number of channels is one, i.e.,  $C = 1$  and  $\gamma$  equals  $\frac{1}{2}$ , the output node representations are as

$$Z^{(l+1)} = \sigma\left(\frac{1}{2} Z^{(l)} W^{(l)} + \frac{1}{2} (\alpha^{(l,1)} \cdot \mathbb{A}) Z^{(l)} W^{(l)}\right). \quad (23)$$

Then if the first FastGT layer only selects the adjacency matrix, i.e.,  $\alpha^{(l,1)} \cdot \mathbb{A} = 1 \cdot A + 0 \cdot I$ , the output representations are exactly same as the output of the GCNs.

Mixhop (Abu-El-Haija et al., 2019) is an extended GNN architecture which can capture long-range dependencies by mixing powers of the adjacency matrix as

$$Z^{(l+1)} = \left\| \sigma(\hat{A}^j Z^{(l)} W_j^{(l)}) \right\|_{j \in P}, \quad (24)$$

where  $P$  is a set of integer adjacency powers,  $\hat{A}$  is a symmetrically normalized adjacency matrix with self-connections, i.e.,  $\hat{A} =$

**Table 1**  
Statistics of both homogeneous and heterogeneous graph datasets.

| Type          | Dataset     | # Nodes   | # Edges    | # Features | # Classes | # Training | # Validation | # Test |
|---------------|-------------|-----------|------------|------------|-----------|------------|--------------|--------|
| Heterogeneous | DBLP        | 18,405    | 67,496     | 334        | 4         | 800        | 400          | 2857   |
|               | ACM         | 8994      | 25,922     | 1902       | 3         | 600        | 300          | 2125   |
|               | IMDB        | 12,772    | 37,288     | 1256       | 3         | 300        | 300          | 2339   |
|               | CS          | 1,116,163 | 28,427,508 | 768        | 3505      | 147,769    | 33,582       | 46,711 |
|               | ML          | 227,144   | 4,249,598  | 768        | 1447      | 19,902     | 6112         | 7911   |
|               | NN          | 66,211    | 845,916    | 768        | 929       | 4745       | 1057         | 2311   |
| Homogeneous   | AIR-USA     | 1190      | 13,599     | 238        | 4         | 119        | 238          | 833    |
|               | BLOGCATALOG | 5196      | 171,743    | 8189       | 6         | 519        | 1039         | 3638   |
|               | CITeseer    | 3327      | 4552       | 3703       | 6         | 120        | 500          | 1000   |
|               | CORA        | 2708      | 5278       | 1433       | 7         | 140        | 500          | 1000   |
|               | FLICKR      | 7575      | 239,738    | 12,047     | 9         | 757        | 1515         | 5303   |
|               | PPI         | 10,076    | 157,213    | 50         | 121       | 1007       | 2015         | 7054   |

$\tilde{D}^{-\frac{1}{2}}(A+I)\tilde{D}^{-\frac{1}{2}}$  and  $\hat{A}^{(j)}$  denotes the adjacency matrix  $\hat{A}$  multiplied by itself  $j$  times. Since the degree matrix of  $A + I$  equals  $2I$ , we can rewrite Eq. (24) as

$$Z^{(l+1)} = \prod_{j \in P} \left( (\alpha \cdot \mathbb{A})^j Z^{(l)} W_j^{(l)} \right), \quad (25)$$

where  $\alpha \cdot \mathbb{A} = \frac{1}{2}A + \frac{1}{2}I$ . Then if  $\gamma$  equals 0, the number of channels equals the size of  $P$  i.e.,  $C = |P|$ , number of FastGT layers in each channel equals  $j$  i.e.,  $K=j$ , all FastGT layers choose the adjacency matrix and the identity matrix in the same ratio, i.e.,  $\alpha^{(l,k)} \cdot \mathbb{A} = \frac{1}{2}A + \frac{1}{2}I$  and a channel aggregation function is an identity function i.e.,  $f_{agg}(x) = x$ , the output is same as the output of the Mixhop, i.e., the Mixhop can be a special case of our FastGTNs.

Lastly, we discuss RGCN (Schlichtkrull et al., 2018) which extends the GCN to heterogeneous graphs by utilizing relation-specific parameters. Specifically, the output representations from the  $l$ th RGCN are as

$$Z^{(l+1)} = \sigma \left( \sum_{t=1}^{|T_e|} D_t^{-1} A_t Z^{(l)} \left( \sum_{i=1}^B a_{ti}^{(l)} V_i^{(l)} \right) \right) \quad (26)$$

$$= \sigma \left( \sum_{i=1}^B a_i^{(l)} \cdot \mathbb{A} Z^{(l)} V_i^{(l)} \right), \quad (27)$$

where  $A_t$  denotes an adjacency matrix for relation type  $t$ ,  $V_i^{(l)}$  denotes a basis parameter of  $l$ th RGCN layer and  $a_{ti}^{(l)}$  denotes a coefficient for relation type  $t$ . The derivation from (26) to (27) is provided in A.3 in the supplement. If the number of FastGT layers is one i.e.,  $K = 1$ , the number of channels equals number of basis matrices i.e.,  $C = B$ ,  $\gamma$  equals 0 and a channel aggregation function  $f_{agg}$  is summation, the output node representations are given as

$$Z^{(l+1)} = \sigma \left( \sum_c \alpha^{(l,1,c)} \cdot \mathbb{A} Z^{(l)} W_c^{(l)} \right). \quad (28)$$

Then (27) and (28) are exactly same except for that (27) apply different linear combinations in each layer.

## 4. Experiments

In this section, we evaluate our proposed methods on both homogeneous and heterogeneous graph datasets. The experiments aim to address the following research questions:

- **Q1.** How effective are the GTNs and FastGTNs with non-local operations compared to state-of-the-art GNNs on both *homogeneous* and *heterogeneous* graphs in node classification?
- **Q2.** Can the FastGTNs *efficiently* perform the *identical* graph transformation compared to the GTNs?

- **Q3.** Can GTNs *adaptively* produce a variable length of meta-paths depending on datasets?
- **Q4.** How can we interpret the importance of each meta-path from the adjacency matrix generated by GTNs?

### 4.1. Experimental settings

**Datasets.** We evaluate our method on twelve benchmark datasets for node classification including six homogeneous graph datasets and six heterogeneous graph datasets for node classification. Detailed statistics regarding each dataset can be found in Table 1. The datasets for each type (i.e., homogeneous or heterogeneous) are as follows:

#### • Heterogeneous Graph Datasets

- DBLP and ACM are both citation networks. They differ in the sense that DBLP has three types of nodes (paper (P), author (A), conference (C)) and four types of edges (PA, AP, PC, CP). ACM is similar but has subject (S) as a node type instead of conference (C), with edge types differing accordingly. DBLP and ACM are node classification datasets with author research area and paper category as labels, respectively.
- IMDB is a movie network dataset. It contains three types of nodes (movies (M), actors (A), directors (D)) and uses the genres of movies as labels.
- CS, ML, NN each refer to domain-specific subgraphs from the Open Academic Graph (OAG) (Tang et al., 2015). OAG is a large citation network with ten types of nodes (paper (P), author (A), field ( $L_0, L_1, L_2, L_3, L_4, L_5$ ), venue (V), institute (I)) and ten types of edges (PA,  $PL_0 - PL_5$ , PV, AI, PP). The task is to predict the venue that each paper is published at.

#### • Homogeneous Graph Datasets

- AIR-USA is a dataset made up of graphs representing airport traffic within the US. Each node represents an airport and edges between nodes indicate the existence of commercial flights between the two airports (Wu, He, & Xu, 2019).
- BLOGCATALOG and FLICKR are both social network datasets, with the former being a blogging platform and the latter being an image and video sharing platform. In both dataset, each node represents a user of the online community and the edges correspond to whether or not users are following each other (Huang, Li, & Hu, 2017).
- CORA and CITESeer are citation network datasets. Both datasets are comprised of nodes which represent papers published in various fields and edges which represent citation links (Kipf & Welling, 2016a).

- PPI refers to the protein–protein interaction network dataset. The network’s nodes represent a protein structure that contains features corresponding to different gene sets. Edges refer to the relation between such proteins (Hamilton et al., 2017).

**Baselines.** To evaluate the effectiveness of representations learnt by the Graph Transformer Networks in node classification, we compare GTNs with conventional random walk based baselines as well as state-of-the-art GNN based methods. Since homogeneous graph neural networks (e.g., GCN, GAT, JK-Net, MixHop and GCNII) cannot differentially handle the different types of nodes and edges, we apply them after converting the heterogeneous graphs into homogeneous graphs, which are the whole graphs without type information.

- **MLP** is a simple baseline model that uses only node features for prediction.
- **Node2Vec** (Grover & Leskovec, 2016) is a random walk based network embedding method which was originally designed for embedding homogeneous graphs. In heterogeneous graphs, we ignore the heterogeneity of nodes and edges and run DeepWalk on the entire heterogeneous graph.
- **GCN** (Kipf & Welling, 2017) utilizes a first-order approximation of the spectral graph filter to aggregate features from neighbors.
- **GAT** (Veličković et al., 2018) leverages an attention mechanism to learn the relative weights between the neighborhood nodes.
- **JK-Net** (Xu et al., 2018) leverages a variable range of neighborhoods by connecting the last layer of the network with all preceding hidden layers.
- **MixHop** (Abu-El-Haija et al., 2019) mixes powers of the adjacency matrices and applies a GCN to capture long-range dependencies.
- **GCNII** (Chen, Wei, Huang, Ding, & Li, 2020) improves GCN with initial residual connection and identity mapping to prevent over-smoothing.
- **RGCN** (Schlichtkrull et al., 2018) employs GCNs with relation-specific weight matrices to deal with heterogeneous graphs.
- **HAN** (Wang, Ji, et al., 2019) uses manually selected meta-paths to transform a heterogeneous graph into a homogeneous graph and then applies GNNs on the homogeneous graph.
- **HGT** (Hu et al., 2020) parameterizes the meta relation triplet of each edge type and uses a structure that utilizes the self-attention of the transformer architecture (Vaswani et al., 2017) to learn specific patterns of different relationships.
- **SimpleHGN** (Lv et al., 2021) adopts GAT with three well-known techniques: learnable edge-type embedding, residual connections, and L2 normalization.
- **GraphMSE** (Li et al., 2021) automatically samples meta-path instances and employs GNN with embeddings of the meta-path instances.
- **LDS-GNN** (Franceschi, Niepert, Pontil, & He, 2019) simultaneously learns the graph structure and the parameters of a GNN by approximately solving a bilevel optimization problem that learns a discrete probability distribution on the edges of the graph.
- **Pro-GNN** (Jin et al., 2020) jointly learns clean graph structure and robust GNN parameters and alternatively updates graph structure by preserving the low rank, sparsity, and feature smoothness properties.

**Implementation details.** All the models in this paper are implemented using PyTorch and PyTorch Geometric (Paszke et al., 2019) and the experiments are conducted on a single GPU (Quadro RTX 8000). For Node2Vec, GCN, GAT, JK-Net, GCNII, and RGCN, we used the implementations in PyTorch Geometric. We re-implemented HAN, HGT, SimpleHGN, GraphMSE, LDS-GNN, and Pro-GNN referencing the code from the authors of the papers (Franceschi et al., 2019; Hu et al., 2020; Jin et al., 2020; Li et al., 2021; Lv et al., 2021; Wang, Ji, et al., 2019). We set the dimensionality of hidden representations to 64 throughout the neural networks and apply the Adam optimizer for all models. For each model and each dataset, we perform a hyper-parameter search within the following ranges: the learning rate is from  $1e-3$  to  $1e-6$ , the dropout rate is from 0.1 to 0.8 and the epoch is from 50 to 200. Based on the accuracy on validation sets, the best models are selected and the models are used for evaluation. From ten independent runs, the mean and standard deviation of micro-F1 scores on test datasets are computed. In our FastGTNs, as discussed in Proposition 1, we perform the row-wise normalization of the adjacency matrices in  $\mathbb{A}$ . To avoid the division-by-zero, we add a small positive number to the diagonal elements of the adjacency matrices, i.e.,  $D^{-1}(A + \epsilon I)$ . Also, weights of  $1 \times 1$  convolution filters  $\{\Phi^{(k)}\}_{k=1}^K$  in each GT layer and FastGT layer are randomly initialized from Gaussian distribution with zero mean and standard deviation of 0.1. In FastGTNs, since we empirically found that the non-local operation is more effective when the weights for the non-local graph are initialized lower than the weights for other graphs, we separately initialize the weights for non-local graphs relatively low, which are selected between 0 and  $-5$  based on the accuracy on validation sets.

#### 4.2. Results on node classification

We evaluated the effectiveness of GTNs and FastGTNs with non-local operations in six heterogeneous graph datasets and six homogeneous graph datasets. By analyzing the result of experiments, we will answer the research Q1.

**Effectiveness of Graph Transformer Networks on heterogeneous graph datasets.** Table 2 and 3 show the classification results on six heterogeneous graph datasets. In large-scale graph datasets (e.g., CS, ML, NN, DBLP, BLOGCATLOG, and FLICKR), we trained GNN-based methods and GTN in the mini-batch setting with graph sampling algorithm (Hamilton et al., 2017; Hu et al., 2020). We observe that our proposed methods, GTN and FastGTN, consistently outperform all network embedding methods and graph neural network methods in five out of six heterogeneous graph datasets. GNN-based methods perform better than random walk-based network embedding methods. Interestingly, though the HAN is a modified GAT for a heterogeneous graph, the GAT usually performs better than the HAN. This result shows that using the pre-defined meta-paths as the HAN may cause adverse effects on performance. In contrast, Our GTN and FastGTN achieved competitive performance compared to all other baselines on all datasets. It demonstrates that the GTN can learn a new graph structure which consists of useful meta-paths for learning more effective node representation. Also, the performance gap between GTNs and FastGTNs on IMDB (60.02% vs. 64.73%) shows that in FastGTNs the graph transformations based on the semantic similarity (i.e., non-local operations) are effective. We additionally provide an ablation study of non-local operations in A.6 in the supplement.

**Effectiveness of Graph Transformer Networks on homogeneous graphs.** In homogeneous graphs, although a number of edge types is only one, as we add an identity matrix to the candidate adjacency matrix, our GTNs can find the effective neighborhood range for each dataset.



**Table 2**  
Node classification performance (NDCG and MRR) on large-scale heterogeneous graph datasets.

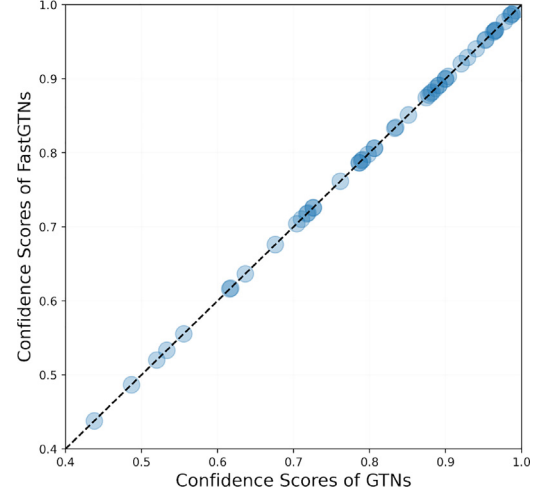
| Model             | CS                 |                    | ML                 |                    | NN                 |                    |
|-------------------|--------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
|                   | NDCG               | MRR                | NDCG               | MRR                | NDCG               | MRR                |
| MLP               | 28.57±0.005        | 12.34±0.004        | 30.89±0.007        | 14.38±0.007        | 25.99±0.002        | 12.03±0.002        |
| Node2Vec          | 26.92±0.005        | 10.93±0.003        | 31.62±0.001        | 15.04±0.001        | 26.16±0.006        | 11.27±0.007        |
| GCN               | 34.72±0.006        | 17.87±0.005        | 37.76±0.005        | 20.66±0.005        | 31.30±0.002        | 15.89±0.003        |
| GAT               | 41.40±0.009        | 24.01±0.008        | 37.83±0.012        | 20.87±0.012        | 29.38±0.007        | 14.03±0.009        |
| RGCN              | 41.83±0.009        | 24.35±0.007        | 39.04±0.004        | 21.71±0.003        | 29.84±0.006        | 14.76±0.007        |
| HGT               | 42.57±0.017        | 25.00±0.017        | 37.31±0.011        | 20.26±0.009        | 28.43±0.016        | 13.70±0.014        |
| SimpleHGN         | 37.59±0.004        | 20.11±0.003        | 38.24±0.004        | 21.18±0.004        | <b>33.27±0.005</b> | <b>17.94±0.005</b> |
| GTN <sub>-I</sub> | 41.81±0.018        | 24.53±0.017        | 40.17±0.014        | 23.27±0.015        | 31.57±0.006        | 16.33±0.003        |
| GTN               | <b>42.75±0.012</b> | <b>25.35±0.011</b> | 39.60±0.016        | 22.70±0.017        | 31.76±0.006        | 16.49±0.008        |
| FastGTN           | 42.49±0.021        | 25.21±0.020        | <b>40.49±0.009</b> | <b>23.52±0.009</b> | 32.12±0.007        | 16.81±0.008        |

**Table 3**  
Node classification (micro F1-score) on heterogeneous graph datasets.

| Model             | DBLP                 | ACM                  | IMDB                 |
|-------------------|----------------------|----------------------|----------------------|
| MLP               | 79.18 ± 0.015        | 86.19 ± 0.003        | 49.51 ± 0.019        |
| Node2Vec          | 86.10 ± 0.001        | 76.27 ± 0.002        | 47.32 ± 0.005        |
| GCN               | 85.63 ± 0.003        | 91.70 ± 0.003        | 60.41 ± 0.009        |
| GAT               | 94.68 ± 0.002        | 91.99 ± 0.003        | 59.64 ± 0.016        |
| RGCN              | 93.16 ± 0.002        | 91.93 ± 0.004        | 59.87 ± 0.008        |
| HAN               | 92.17 ± 0.005        | 91.10 ± 0.004        | 59.80 ± 0.013        |
| HGT               | 94.21 ± 0.005        | 91.14 ± 0.005        | 60.98 ± 0.002        |
| GraphMSE          | 94.19 ± 0.003        | 92.93 ± 0.004        | 56.40 ± 0.016        |
| SimpleHGN         | 94.34 ± 0.006        | 92.30 ± 0.007        | 61.50 ± 0.013        |
| GTN <sub>-I</sub> | 94.74 ± 0.005        | 85.36 ± 0.021        | 59.27 ± 0.021        |
| GTN               | 94.47 ± 0.003        | 91.96 ± 0.005        | 61.02 ± 0.018        |
| FastGTN           | <b>94.88 ± 0.004</b> | <b>92.94 ± 0.003</b> | <b>64.74 ± 0.017</b> |

Table 4 shows the performance of GTNs, FastGTNs and other baselines in homogeneous graphs. We additionally compared our methods with three well-known GNN models (MixHop (Abu-El-Haija et al., 2019), JK-Net (Xu et al., 2018) and GCNII (Chen et al., 2020)) and two graph structure learning methods (LDS-GNN (Franceschi et al., 2019) and Pro-GNN (Jin et al., 2020)). LDS-GNN (Franceschi et al., 2019) and Pro-GNN (Jin et al., 2020) were not evaluated on PPI dataset since they were not designed for multi-label classification. We can observe that our GTN and FastGTN consistently outperform all GNN baselines in five datasets, especially on BLOGCATALOG and FLICKR datasets. Interestingly, in BLOGCATALOG and FLICKR datasets, the Multi-Layer Perceptron (MLP) model using the *only* node features achieved better performance than all GNN baseline models. It implies that noisy input graphs rather hinder learning of most GNNs and even other graph structure learning methods (LDS-GNN and Pro-GNN) whereas GTNs and FastGTNs successfully suppress the noisy edges by weighing an attention score of an identity matrix and learn a high accuracy classifier.

**Identity matrix in  $\mathbb{A}$  to learn variable-length meta-paths.** As mentioned in Section 3.2, the identity matrix is included in the candidate adjacency matrices  $\mathbb{A}$ . To verify the effect of identity matrix, we trained and evaluated another model named  $GTN_{-I}$  as an ablation study. the  $GTN_{-I}$  has exactly the same model structure as GTN but its candidate adjacency matrix  $\mathbb{A}$  does not include an identity matrix. In general, the  $GTN_{-I}$  usually performs worse than the GTN. In heterogeneous graph datasets, it is worth to note that the difference is greater in IMDB than DBLP. One explanation is that the length of meta-paths  $GTN_{-I}$  produced is not effective in IMDB. As we stacked 3 layers of GTL,  $GTN_{-I}$  always produce 4-length meta-paths. However shorter meta-paths (e.g. MDM) are preferable in IMDB. Also, in homogeneous graph datasets, the differences in BLOGCATALOG and FLICKR are extremely big, which are 48% and 150%. It demonstrates that including identity matrix is effective in learning GTNs on noisy graphs.

**Fig. 4.** Confidence scores of GTNs (x-axis) and FastGTNs (y-axis). On 50 random samples from the test set of IMDB, the confidence scores of GTNs and FastGTNs are practically identical. All 50 points are on the Identity line i.e.,  $y = x$ .

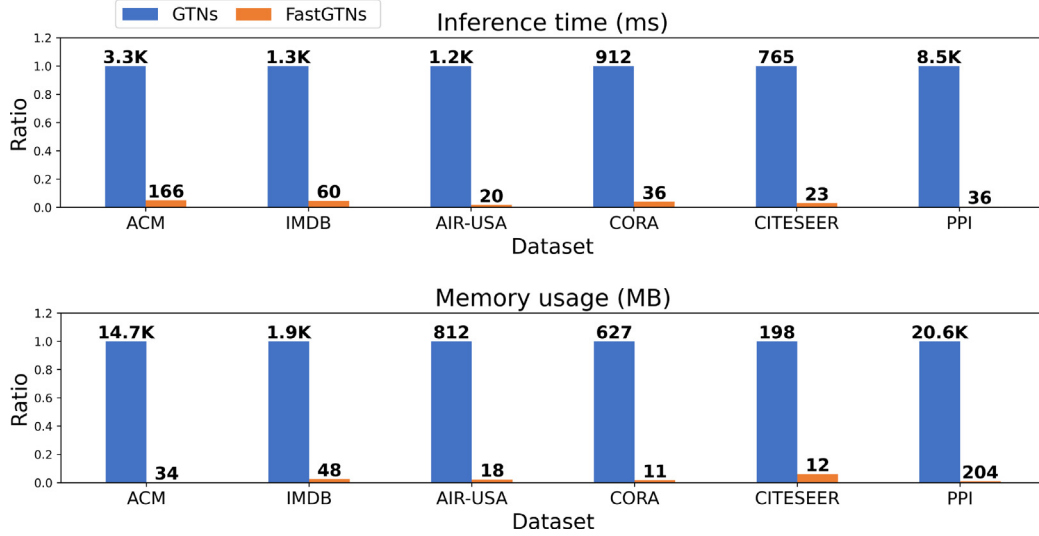
#### 4.3. Exactness and efficiency of FastGTNs

In this section, we show the exactness and efficiency of FastGTN compared to GTN. First, As discussed in (11) in Section 3.4, Our FastGTN without non-local operations is an exact version of GTN. It means that the model parameters from GTN are compatible with FastGTN and the graph transformations of GTN can be *identically* performed by FastGTN. To show the exactness of FastGTN, we first train a GTN with  $K$  GT layers on the IMDB dataset and copy the model parameters of the GTN ( $\{\Phi^{(k)}\}_{k=1}^K$ ) and the GNN  $\{W^{(l)}\}_{l=1}^L$  to the corresponding model parameters in our FastGTN with  $(K+1)$  FastGT layers. Note that unlike other GT layers, the first GT layer has two convex combinations of adjacency matrices. So to represent the same transformation, two FastGT layers are needed for the first GT layer. This is why we need  $K+1$  FastGT layers. Fig. 4 proves that the predictions (confidence scores) by the FastGTN and the GTN are identical. All 50 randomly drawn data points from a test set are on the Identity line (i.e.,  $y = x$ ). Note that the GTN parameters  $\{\Phi^{(k)}\}_{k=1}^K$  should be reversely copied as in (14). As our FastGTNs also generalize of other popular graph neural networks such as GCN and MixHop, we prove that predictions (confidence scores) by special cases of our FastGTNs and the two GNNs are identical (see A.5 in the supplement).

Second, the main contribution of our FastGTN is improving the efficiency of graph transformations. For more detailed efficiency comparisons with GTN, we measured the inference time and memory consumption of the two methods. Fig. 5 shows that

**Table 4**  
Node classification performance (micro F1-score) on homogeneous graph datasets.

| Model             | AIR-USA              | BLOGC                | CITeseer             | CORA                 | FLICKR               | PPI                  |
|-------------------|----------------------|----------------------|----------------------|----------------------|----------------------|----------------------|
| MLP               | 55.10 ± 0.006        | 82.42 ± 0.010        | 59.23 ± 0.007        | 55.78 ± 0.014        | 69.10 ± 0.009        | 40.38 ± 0.001        |
| Node2Vec          | 45.86 ± 0.013        | 61.23 ± 0.001        | 33.51 ± 0.007        | 54.30 ± 0.006        | 46.50 ± 0.002        | 40.87 ± 0.001        |
| GCN               | 57.30 ± 0.009        | 75.25 ± 0.006        | 68.42 ± 0.006        | 79.65 ± 0.005        | 52.95 ± 0.009        | 42.56 ± 0.003        |
| GAT               | 53.06 ± 0.001        | 55.63 ± 0.021        | 68.92 ± 0.006        | 79.85 ± 0.009        | 36.10 ± 0.017        | 40.63 ± 0.013        |
| JK-NET            | 57.15 ± 0.009        | 68.47 ± 0.004        | 68.11 ± 0.007        | 79.57 ± 0.006        | 54.01 ± 0.006        | 42.38 ± 0.003        |
| MixHop            | 55.15 ± 0.011        | 66.96 ± 0.003        | 67.74 ± 0.013        | 79.32 ± 0.007        | 48.91 ± 0.014        | 42.52 ± 0.002        |
| GCNII             | 56.25 ± 0.010        | 64.17 ± 0.004        | 68.11 ± 0.017        | 79.24 ± 0.014        | 33.71 ± 0.008        | 42.36 ± 0.004        |
| LDS-GNN           | 37.15 ± 0.023        | 21.16 ± 0.016        | <b>73.60 ± 0.068</b> | 79.72 ± 0.055        | 14.07 ± 0.090        | N/A                  |
| Pro-GNN           | 56.10 ± 0.054        | OOM                  | 68.85 ± 0.063        | 78.80 ± 0.073        | OOM                  | N/A                  |
| GTN <sub>-l</sub> | 61.51 ± 0.013        | 60.73 ± 0.009        | 64.64 ± 0.009        | 76.97 ± 0.005        | 30.65 ± 0.008        | 42.95 ± 0.004        |
| GTN               | <b>61.80 ± 0.008</b> | 90.30 ± 0.006        | 68.68 ± 0.011        | 79.99 ± 0.008        | 76.77 ± 0.009        | 42.64 ± 0.003        |
| FastGTN           | 58.39 ± 0.011        | <b>90.80 ± 0.005</b> | 69.42 ± 0.009        | <b>80.69 ± 0.009</b> | <b>77.05 ± 0.009</b> | <b>43.29 ± 0.011</b> |



**Fig. 5.** Inference time (up) and memory usage (down) of GTNs (blue) and our FastGTNs (orange) on both homogeneous and heterogeneous graph datasets. To show how efficiently FastGTNs can perform the *identical* graph transformations, FastGTNs were measured without the non-local operations. FastGTNs significantly speed up the graph transformations and reduce the memory usage in all datasets. Especially, on a large-scale graph dataset (PPI) FastGTNs show 230× faster inference time and 100× less memory usage than the GTNs. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

FastGTN is significantly more efficient than GTN in both the inference time and memory consumption. The performance gain is larger on larger graphs. In particular, on the PPI dataset, our experiments show that our FastGTN is 230× faster and 100× more memory-efficient than the GTN during inference. Again, this speed-up and memory efficiency are achieved without any accuracy loss. During training, a similar performance gain is observed (see A.4 in the supplement).

#### 4.4. Interpretation of graph transformer networks

We examine the transformation learnt by GTNs to discuss the question interpretability **Q4**. We first describe how to calculate the importance of each meta-path from our GT layers. For the simplicity, we assume the number of output channels is one. Then, the new adjacency matrix from the  $l$ th GT layer can be written as

$$\begin{aligned}
 A^{(k)} &= (D^{(k-1)})^{-1} \dots (D^{(1)})^{-1} \left( (\alpha^{(0)} \cdot \mathbb{A}) (\alpha^{(1)} \cdot \mathbb{A}) \dots (\alpha^{(k)} \cdot \mathbb{A}) \right) \\
 &= (D^{(k-1)})^{-1} \dots (D^{(1)})^{-1} \\
 &\quad \left( \sum_{t_0, t_1, \dots, t_k \in \mathcal{T}^e} \alpha_{t_0}^{(0)} \alpha_{t_1}^{(1)} \dots \alpha_{t_k}^{(k)} A_{t_0} A_{t_1} \dots A_{t_k} \right),
 \end{aligned} \tag{29}$$

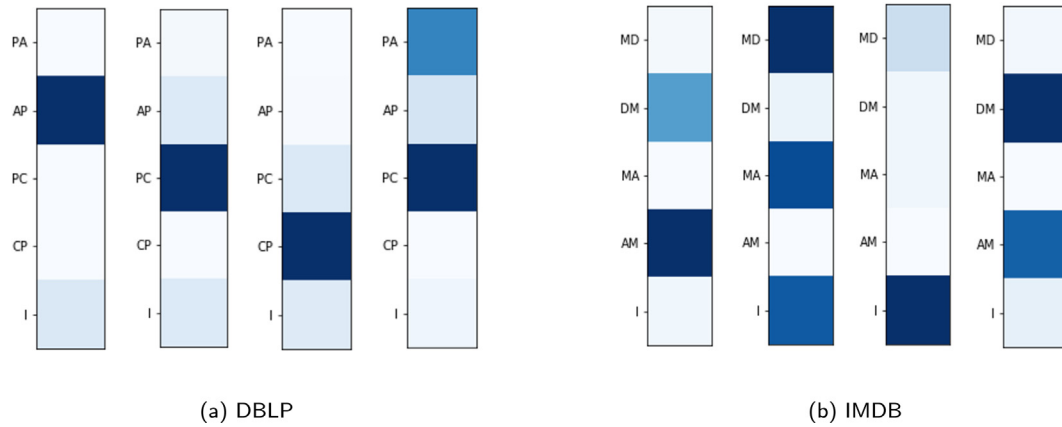
**Table 5**

Comparison with predefined meta-paths and top-ranked meta-paths by GTNs. Our model found important meta-paths that are consistent with pre-defined meta-paths between target nodes (a type of nodes with labels for node classifications). Also, new relevant meta-paths between all types of nodes are discovered by GTNs.

| Dataset | Predefined meta-path | Meta-path learnt by GTNs     |                  |
|---------|----------------------|------------------------------|------------------|
|         |                      | Top 3 (between target nodes) | Top 3 (all)      |
| DBLP    | APCPA, APA           | APCPA, APAPA, APA            | CPCPA, APCPA, CP |
| ACM     | PAP, PSP             | PAP, PSP                     | APAP, APA, SPAP  |
| IMDB    | MAM, MDM             | MDM, MAM, MDMDM              | DM, AM, MDM      |

where  $\mathcal{T}^e$  denotes a set of edge types and  $\alpha_{t_l}^{(l)}$  is an attention score for edge type  $t_l$  at the  $l$ th GT layer. So,  $A^{(l)}$  can be viewed as a weighted sum of all meta-paths including 1-length (original edges) to  $l$ -length meta-paths. The contribution of a meta-path  $t_l, t_{l-1}, \dots, t_0$  is obtained by  $\prod_{i=0}^l \alpha_{t_i}^{(i)}$ .

Now we can interpret new graph structures learnt by GTNs. The weight  $\prod_{i=0}^l \alpha_{t_i}^{(i)}$  for a meta-path  $(t_0, t_1, \dots, t_l)$  is an attention score and it provides the importance of the meta-path in the prediction task. In Table 5 we summarized predefined meta-paths, that are widely used in literature, and the meta-paths with high attention scores learnt by GTNs.



**Fig. 6.** After applying softmax function on  $1 \times 1$  conv filter  $\phi_i^{(k)}$  ( $k$ : index of layer) in Fig. 1, we visualized this attention score of adjacency matrix (edge type) in DBLP (left) and IMDB (right) datasets. (a) Respectively, each edge indicates (Paper-Author), (Author-Paper), (Paper-Conference), (Conference-Paper), and identity matrix. (b) Edges in IMDB dataset indicates (Movie-Director), (Director-Movie), (Movie-Actor), (Actor-Movie), and identity matrix.

As shown in Table 5, between target nodes, that have class labels to predict, the predefined meta-paths by domain knowledge are consistently top-ranked by GTNs as well. This shows that GTNs are capable of learning the importance of meta-paths for tasks. More interestingly, GTNs discovered important meta-paths that are not in the predefined meta-path set. For example, in the DBLP dataset GTN ranks CPCPA as most importance meta-paths, which is not included in the predefined meta-path set. It makes sense that author's research area (label to predict) is relevant to the venues where the author publishes. We believe that the interpretability of GTNs provides useful insight in node classification by the attention scores on meta-paths.

Fig. 6 shows the attention scores of adjacency matrices (edge type) from each Graph Transformer Layer. Compared to the result of DBLP, identity matrices have higher attention scores in IMDB. As discussed in Section 3.3, a GTN is capable of learning shorter meta-paths than the number of GT layers, which they are more effective as in IMDB. By assigning higher attention scores to the identity matrix, the GTN tries to stick to the shorter meta-paths even in the deeper layer. This result demonstrates that the GTN has ability to adaptively learns most effective meta-path length depending on the dataset. Also, we additionally provide an interpretation of GTNs in homogeneous graphs in A.7 in the supplement.

## 5. Conclusion

We proposed Graph Transformer Networks for learning node representations on both homogeneous graphs and heterogeneous graphs. Our approach transforms graphs into multiple new graphs defined by meta-paths with arbitrary edge types and arbitrary length up to one less than the number of Graph Transformer layers while it learns node representation via convolution on the learnt meta-path graphs. Also, we proposed the enhanced version of GTNs, Fast Graph Transformer Networks, which are up to  $230 \times$  and  $150 \times$  faster in inference and training, and use up to  $100 \times$  and  $148 \times$  less memory while allowing the *identical* graph transformations as GTNs. The learnt graph structures from GTNs and FastGTNs lead to more effective node representation resulting in state-of-the art performance, without any predefined meta-paths from domain knowledge, on all twelve benchmark node classification on both homogeneous and heterogeneous graphs.

Since our Graph Transformer layers can be combined with existing GNNs, we believe that our framework opens up new ways for GNNs to optimize graph structures by themselves to operate convolution depending on data and tasks without any

manual efforts. As several heterogeneous graph datasets have been recently studied for other network analysis tasks, such as link prediction (Wang, He, Cao, Liu & Chua, 2019; Zhang, Song, Huang, Swami, & Chawla, 2019) and graph classification (Kim et al., 2019; Linmei, Yang, Shi, Ji, & Li, 2019), applying our GTNs to the other tasks can be interesting future directions.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgments

This research was supported by the following funding sources: National Research Foundation of Korea (NRF-2020R1A2 C3010638); ICT Creative Consilience program (IITP-2022-2020-0-01819) and Research on CPU vulnerability detection and validation (No.2019-0-00533) supervised by the IITP (Institute for Information & communications Technology Planning & Evaluation) funded by the Korea government (MSIT).

## Appendix

### A.1. Mini-batch training for GTNs

Since GTNs transform the entire input graph *at once*, when the size of an input graph is too large, GTNs requires an excessive amount of memory and incurs high computational cost. To alleviate this scalability issue, we present a mini-batch training algorithm for GTNs. Algorithm 1 describes our mini-batch training algorithm. Specifically, we first combine a set of input adjacency matrices  $\{A_t\}_{t=1}^{T_e}$  into one adjacency matrix  $A$  for extracting sub-graphs. We then select target nodes for each mini-batch training iteration. For each iteration, based on the target nodes of each mini-batch, we extract a subgraph and use its adjacency matrix  $A'$  from the original graph's adjacency matrix  $A$  by using two types of graph sampling algorithms: a neighborhood-based graph sampling algorithm (Hamilton et al., 2017) and a layer-wise graph sampling algorithm (Hu et al., 2020). After sampling the subgraph, we separate the sampled subgraph's adjacency matrix  $A'$  into a set of adjacency matrices corresponding to each edge type such as  $\{A'_t\}_{t=1}^{T_e}$ . The sampled adjacency matrix set  $\{A'_t\}_{t=1}^{T_e}$  is fed into GTNs for mini-batch training. This mini-batch training algorithm

enables GTNs to handle large-scale graph datasets with up to 30 million edges in an efficient manner.

---

**Algorithm 1:** Mini-batch training algorithm for Graph Transformer Networks

---

**Input:** set of adjacency matrices  $\mathbb{A}$ ; feature matrix  $X$ ; training set  $\mathcal{V}_{train}$  and  $\mathcal{Y}_{train}$ ; Graph Transformer Networks  $f_\theta$ ; number of layers  $L$ ;  
**Output:** set of adjacency matrices  $\mathbb{A}'$

- 1 Combine a set of input adjacency matrices into one adjacency matrix  $A \leftarrow \cup_{t=1}^T A_t$ ;
- 2 **while do**
- 3   Sample a mini-batch of  $m$  target nodes  $\{v_i\}_{i=1}^m$  from the training set  $\mathcal{V}_{train}$  with corresponding targets  $Y_B$ ;
- 4    $\mathcal{V}_B \leftarrow \{v_i\}_{i=1}^m$ ;
- 5    $\mathcal{E}_B \leftarrow \phi$ ;
- 6   **for**  $l = 1, 2, \dots, L + 1$  **do**
- 7     Sample nodes  $\mathcal{V}^{(l)}$  and edges  $\mathcal{E}^{(l)}$  based on  $\mathcal{V}_B$  by using graph sampling algorithm (Hamilton et al., 2017; Hu et al., 2020);
- 8      $\mathcal{V}_B \leftarrow \mathcal{V}_B \cup \mathcal{V}^{(l)}$ ;
- 9      $\mathcal{E}_B \leftarrow \mathcal{E}_B \cup \mathcal{E}^{(l)}$ ;
- 10   Reconstruct an adjacency matrix  $A'$  based on sampled nodes  $\mathcal{V}_B$  and edges  $\mathcal{E}_B$ ;
- 11   Divide the adjacency matrix  $A'$  into a set of adjacency matrices corresponding to each edge type  $\{A'_t\}_{t=1}^T$ ;
- 12   Compute prediction  $Y'$  from  $f_\theta$ ,  $X$  and  $A'$  by using Eq. (8) and the node classifier;
- 13   Calculate cross-entropy loss from  $Y'$  and  $Y_B$ ;
- 14   Update weights of  $f_\theta$ ;

---

#### A.2. Proofs of Proposition 1

**Proposition 1.** Given two normalized adjacency matrices  $A, B \in \mathbf{R}^{N \times N}$ , the followings are equivalent:

- (i)  $(D_A^{-1}A)(D_B^{-1}B) = (D_{AB}^{-1}AB)$
- (ii)  $D_{AB}^{-1} = I$
- (iii)  $D_{A+I}^{-1} = (D_A + I)^{-1} = \frac{1}{2}I$

Since the adjacency matrices  $A, B$  are normalized, i.e.,  $\sum_j A_{ij} = 1$ , the degree matrices  $D_A, D_B$  are equal to the identity matrix ( $I$ ) and the inverse degree matrices  $D_A^{-1}, D_B^{-1}$  are also equal to the identity matrix ( $I$ ) respectively. Thus, (i) in Proposition 1 can be re-written as  $AB = D_{AB}^{-1}AB$  and to satisfy the (i), we need to prove that  $D_{AB}$  is an identity matrix. Since  $D_{AB}$  is the degree matrix of the multiplication of two matrices  $A$  and  $B$ , each  $i$ th diagonal element of  $D_{AB}$  can be represented as  $D_{AB}[i, i] = \sum_j (AB)_{ij} = \sum_j \sum_k A_{ik}B_{kj}$ . Then, we can derive that  $D_{AB}$  is equal to the identity matrix, i.e.,  $D_{AB}[i, i] = 1$  as follows:

$$\begin{aligned}
 D_{AB}[i, i] &= \sum_j (AB)_{ij} \\
 &= \sum_j \sum_k A_{ik}B_{kj} \\
 &= \sum_k \sum_j A_{ik}B_{kj} && \because \text{commutativity of sum} \\
 &= \sum_k A_{ik} \sum_j B_{kj} \\
 &= \sum_k \left( A_{ik} \sum_j B_{kj} \right)
 \end{aligned}$$

$$\begin{aligned}
 &= \sum_k A_{ik} && \because \sum_j B_{kj} = 1 \\
 &= 1 && \because \sum_k A_{ik} = 1
 \end{aligned}$$

Therefore, degree matrix  $D_{AB}$  is equal to an identity matrix  $I$ , which satisfies (i), (ii) and (iii) in Proposition 1.

#### A.3. Relation to RGCN

If input graphs are normalized, RGCN (Schlichtkrull et al., 2018) can be subsumed by our FastGTNs with minor modifications. The RGCN (Schlichtkrull et al., 2018) extends the GCN to heterogeneous graphs by utilizing relation-specific parameters. Specifically, the output representations from the  $l$ th RGCN are as

$$Z^{(l+1)} = \sigma \left( \sum_{t=1}^{|\mathcal{T}_e|} D_t^{-1} A_t Z^{(l)} W_t^{(l)} \right), \quad (30)$$

where  $A_t$  denotes an adjacency matrix of  $t$ th edge type  $W_t^{(l)}$  denotes the relation specific parameters of the model. The RGCN also address overparameterization by proposing basis decomposition of  $W_t^{(l)}$  as  $W_t^{(l)} = \sum_{b=1}^B a_{tb}^{(l)} V_b^{(l)}$ , consequently the equation of the RGCN is re-written as

$$Z^{(l+1)} = \sigma \left( \sum_{t=1}^{|\mathcal{T}_e|} D_t^{-1} A_t Z^{(l)} \left( \sum_{i=1}^B a_{ti}^{(l)} V_i^{(l)} \right) \right), \quad (31)$$

Then to compare with our FastGTNs, we derive the equation similar to our FastGTNs as follows:

$$Z^{(l+1)} = \sigma \left( \sum_{t=1}^{|\mathcal{T}|} D_t^{-1} A_t Z^{(l)} \left( \sum_{i=1}^B a_{ti}^{(l)} V_i^{(l)} \right) \right) \quad (32)$$

$$= \sigma \left( \sum_{t=1}^{|\mathcal{T}|} A_t Z^{(l)} \left( \sum_{i=1}^B a_{ti}^{(l)} V_i^{(l)} \right) \right) \quad (33)$$

$$= \sigma \left( \sum_{t=1}^{|\mathcal{T}|} \sum_{i=1}^B A_t Z^{(l)} a_{ti}^{(l)} V_i^{(l)} \right) \quad (34)$$

$$= \sigma \left( \sum_{i=1}^B \sum_{t=1}^{|\mathcal{T}|} A_t Z^{(l)} a_{ti}^{(l)} V_i^{(l)} \right) \quad (35)$$

$$= \sigma \left( \sum_{i=1}^B \sum_{t=1}^{|\mathcal{T}|} a_{ti}^{(l)} A_t Z^{(l)} V_i^{(l)} \right) \quad (36)$$

$$= \sigma \left( \sum_{b=1}^B \left( \sum_{t=1}^{|\mathcal{T}|} a_{ti}^{(l)} A_t \right) Z^{(l)} V_i^{(l)} \right) \quad (37)$$

$$= \sigma \left( \sum_{i=1}^B a_i^{(l)} \cdot \mathbb{A} Z^{(l)} V_i^{(l)} \right) \quad (38)$$

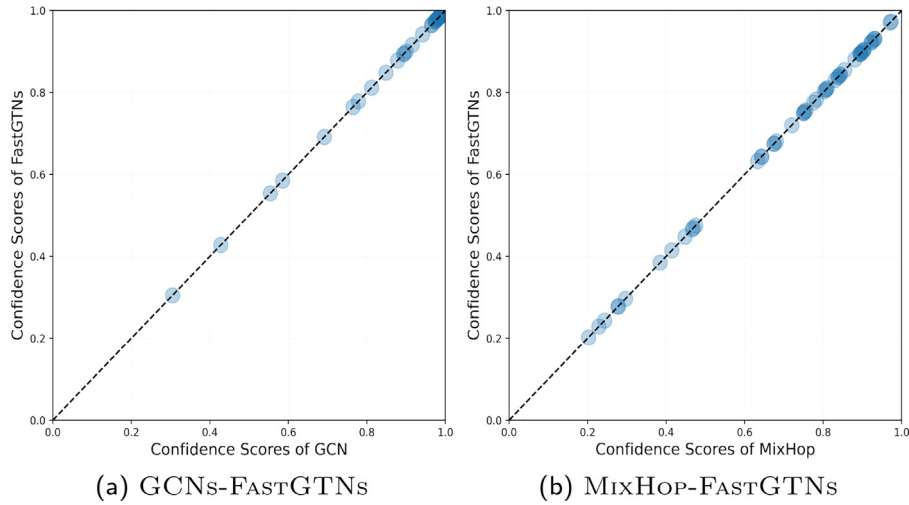
#### A.4. Training efficiency

In this section, we compared our FastGTNs with GTNs in terms of the training efficiency. As shown in Fig. 7, we measured the training time and the memory consumption of the two methods. Fig. 7 shows that FastGTNs are significantly more efficient than GTNs in terms of both the training time and memory. In particular, the comparison on the large-scale dataset, PPI, shows our FastGTNs are 150× faster and 60× more efficient than the GTNs.





**Fig. 7.** Comparisons of training time (up) and memory usage (down) between GTNs (blue) and FastGTNs (orange) on both homogeneous and heterogeneous graph datasets (x-axis). For fair comparison, FastGTNs were measured without the non-local operations. FastGTNs significantly speed up the graph transformations  $150\times$  (PPI) and reduce the memory usage  $60\times$  (PPI). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)



**Fig. 8.** We visualized correlations of confidence scores between other GNNs ((a) GCNs and (b) MixHop) and FastGTNs on 50 randomly drawn data points from a test set of CORA dataset.

#### A.5. Generalization of GCN and MixHop

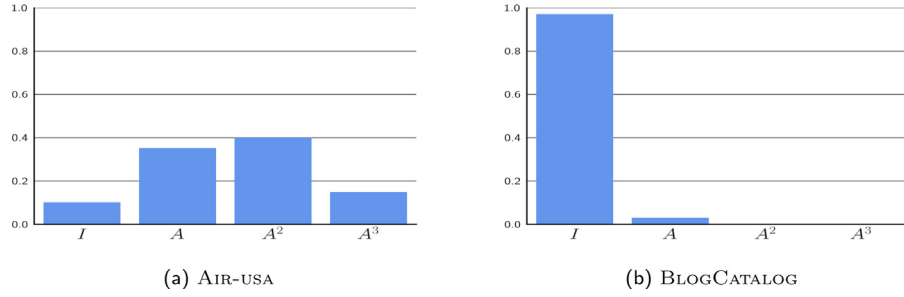
As discussed in Section 3.6, our FastGTNs subsume two popular graph neural networks, GCN and MixHop. For the GCN, if the number of FastGT layers in FastGTNs is one i.e.,  $K = 1$ , the number of channels is one, i.e.,  $C = 1$ ,  $\gamma$  equals  $\frac{1}{2}$  and the first FastGT layer only selects the adjacency matrix, i.e.,  $\alpha^{(1)} \cdot \mathbb{A} = 1 \cdot A + 0 \cdot I$ , the GCN can be a special case of FastGTNs. For the MixHop, if  $\gamma$  equals 0, the number of channels equals the size of  $P$  i.e.,  $C = |P|$ , number of FastGT layers in each channel equals  $j$  and all FastGT layers choose the adjacency matrix and the identity matrix in the same ratio, i.e.,  $\alpha^{(k)} \cdot \mathbb{A} = \frac{1}{2}A + \frac{1}{2}I$ , the output is same as the output of the Mixhop, i.e., the Mixhop can be a special case of FastGTNs. To show that special cases of FastGTNs can be identically performed by the GCN and the MixHop, we first train a GCN and a MixHop on the CORA dataset and copy the model parameters of  $\text{GCN}(\{W_j^{(l)}\}_{l=1}^L)$  and  $\text{MixHop}(\{W_j^{(l)}\}_{l=1}^L)$  to the corresponding model parameters in our FastGTN. Fig. 8 proves

that the predictions (confidence scores) by the FastGTN and other graph neural networks, GCN and MixHop, are identical. All 50 randomly drawn data points from a test set are on the Identity line (i.e.,  $y = x$ ).

#### A.6. Ablation study for non-local operations

We evaluate the effectiveness of non-local operations in FastGTNs.

Table 6 shows the performance gap of FastGTNs with/without non-local operations. Also, the attention scores on the non-local adjacency matrix at each layer of FastGTNs are reported to show whether the non-local operations are adaptively applied. First, non-local operations improve the performance in all datasets by  $0.42 \sim 4.51$  in terms of the micro-F1 score. In addition, the attention scores in Table 6 shows that FastGTNs can adaptively exploit non-local operations by adjusting the attention scores depending on datasets.



**Fig. 9.** We visualized the ratios corresponding to each hop in the output matrix  $A^{(l)}$  from GTNs based on the attention scores of each GT Layer in AIR-USA (left) and BLOGCATALOG (right) dataset. In (a), we use two GT layers in GTNs, thus output matrix from GTNs can involve up to three hop adjacency matrix. In (b), we use one GT layer, thus output matrix from GTNs can involve up to two hop adjacency matrix.

**Table 6**

Ablation study for non-local operations on both homogeneous and heterogeneous graph datasets. Non-local operations improve the performance of FastGTNs on all datasets. The attention scores on non-local operations show that our FastGTNs adaptively leverage non-local operations by adjusting the attention scores on non-local adjacency matrices.

| Dataset | w/o non-local | w/ non-local                    | Attention scores on non-local op. |       |
|---------|---------------|---------------------------------|-----------------------------------|-------|
|         |               |                                 | L1                                | L2    |
| BLOGC   | 88.96         | <b>90.80</b> ( $\uparrow$ 1.84) | 0.012                             | 0.005 |
| FLICKR  | 75.01         | <b>77.05</b> ( $\uparrow$ 2.04) | 0.004                             | 0.005 |
| AIR-USA | 56.60         | <b>58.39</b> ( $\uparrow$ 1.79) | 0.153                             | 0.158 |
| CITSEER | 68.32         | <b>69.42</b> ( $\uparrow$ 1.10) | 0.099                             | 0.095 |
| CORA    | 79.17         | <b>80.69</b> ( $\uparrow$ 1.52) | 0.146                             | 0.152 |
| PPI     | 40.95         | <b>43.29</b> ( $\uparrow$ 2.34) | 0.243                             | 0.223 |
| DBLP    | 94.46         | <b>94.88</b> ( $\uparrow$ 0.42) | 0.048                             | 0.049 |
| ACM     | 91.79         | <b>92.94</b> ( $\uparrow$ 1.15) | 0.026                             | 0.026 |
| IMDB    | 60.23         | <b>64.74</b> ( $\uparrow$ 4.51) | 0.066                             | 0.069 |

**Table 7**

Ablation study for # FastGTN layers and # FastGT layers on Cora and IMDB datasets. The effective number of FastGTN layers is different for each dataset, which showed the best performance on Cora dataset when  $L$  is 1 and on IMDB dataset when  $L$  is 2.

| # FastGTN Layers | # FastGT Layers | CORA                               | IMDB                               |
|------------------|-----------------|------------------------------------|------------------------------------|
| 1                | 1               | 76.06 $\pm$ 0.0080                 | 62.80 $\pm$ 0.0235                 |
|                  | 2               | 79.74 $\pm$ 0.0097                 | 63.66 $\pm$ 0.0164                 |
|                  | 3               | <b>81.27<math>\pm</math>0.0057</b> | 64.04 $\pm$ 0.0130                 |
|                  | 4               | 81.21 $\pm$ 0.0083                 | 62.18 $\pm$ 0.0157                 |
|                  | 5               | 81.19 $\pm$ 0.0072                 | 62.63 $\pm$ 0.0256                 |
|                  | 6               | 81.01 $\pm$ 0.0087                 | 64.04 $\pm$ 0.0178                 |
| 2                | 1               | 79.58 $\pm$ 0.0104                 | 64.18 $\pm$ 0.0183                 |
|                  | 2               | 81.05 $\pm$ 0.0085                 | 63.98 $\pm$ 0.0146                 |
|                  | 3               | 79.75 $\pm$ 0.0061                 | <b>64.62<math>\pm</math>0.0150</b> |
| 3                | 1               | 79.24 $\pm$ 0.0135                 | 63.46 $\pm$ 0.0150                 |
|                  | 2               | 78.68 $\pm$ 0.0092                 | 62.45 $\pm$ 0.0175                 |

#### A.7. Interpretation of GTNs in homogeneous graphs

Fig. 9 shows ratios of each power of adjacency matrix in the output matrix  $A^{(K)}$  from GTNs, respectively in the AIR-USA (left) and BLOGCATALOG (right) datasets. Based on Eq. (30), we calculate the ratio of each hop in the final adjacency matrix after GTNs. Specifically, if the number of GT layers equals two, then the ratio of the identity matrix  $I$  is obtained by  $(\alpha_I^{(0)} * \alpha_I^{(1)} * \alpha_I^{(2)})$  and the ratio of the adjacency matrix  $A$  is obtained by  $(\alpha_A^{(0)} * \alpha_I^{(1)} * \alpha_I^{(2)} + \alpha_I^{(0)} * \alpha_A^{(1)} * \alpha_I^{(2)} + \alpha_I^{(0)} * \alpha_I^{(1)} * \alpha_A^{(2)})$ . As we discussed in Section 4.2, since graph structures in the BLOGCATALOG dataset are noisy enough for a simple MLP to outperform other GNN baselines, GTNs learn to assign higher attention scores to the identity matrix, which effectively minimizes the range of neighborhoods. In contrast, in the AIR-USA dataset, each GT layer assigns relatively higher attention scores to the adjacency matrix, which expands the range of neighborhoods. GTNs can adaptively learn effective range of neighborhoods depending on the dataset.

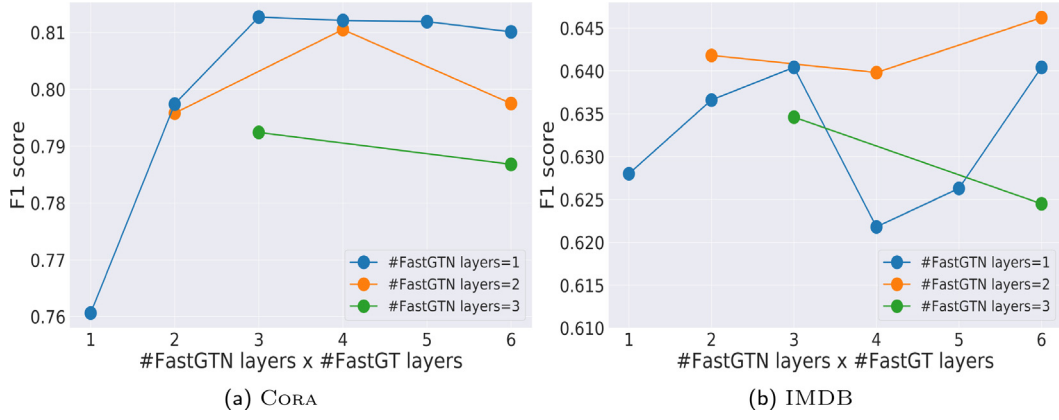
#### A.8. Ablation study of FastGTN layers ( $L$ ) and FastGT layers ( $K$ )

To examine the effectiveness of stacking FastGT layers ( $K$ ) and FastGTN layers ( $L$ ), we did the ablation study regarding  $K$  and  $L$ . We considered ranges of  $K$  and  $L$  up to the case where the number of total layers ( $K * L$ ) is up to 6. As shown in Table 7 and Fig. 10, FastGTNs achieved the best performance on the Cora dataset when  $L$  is 1 and on the IMDB dataset when  $L$  is 2. Also, in cases where #total layers ( $K * L$ ) is same but  $L$  is different (e.g., ( $L=1, K=2$ ) and ( $L=2, K=1$ )), performance is always the best when  $L$  is 1 in Cora dataset whereas  $L$  is 2 in IMDB dataset. This shows that

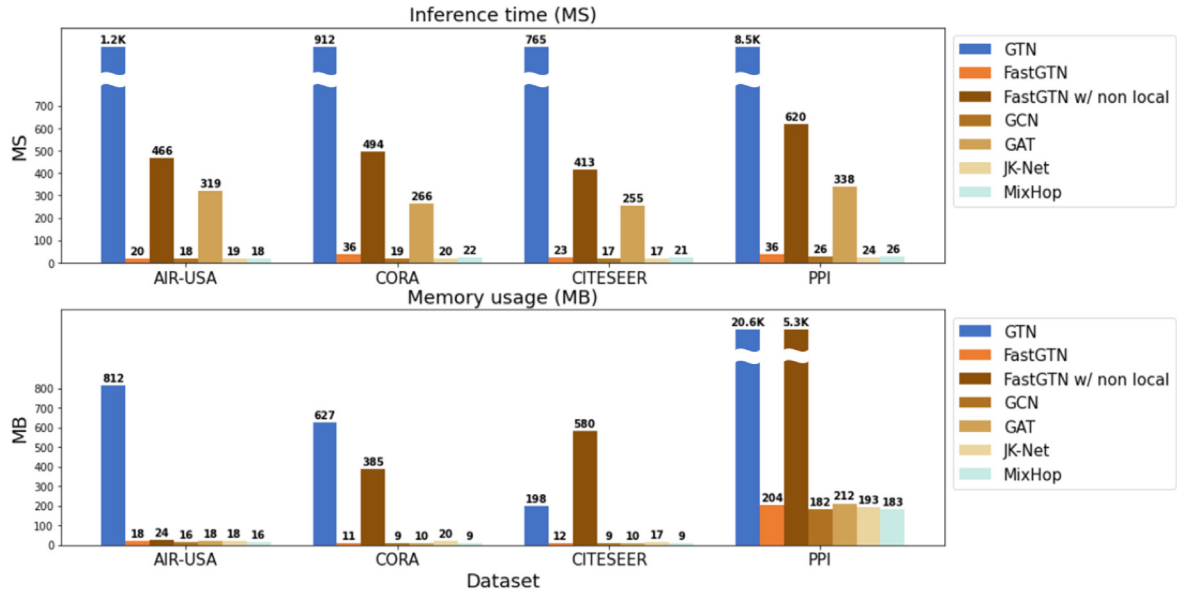
the effective number of FastGTN layers differs depending on each dataset. When #FastGTN layer ( $L$ ) is 1, each channel in a FastGTN layer transforms the graph independently of each other, whereas when the FastGTN layer is more than 1, graph transformation is performed based on the information of aggregating outputs from channels in the previous FastGTN layer. We conjecture that the aggregated outputs is key to the improvement of FastGTNs in IMDB dataset.

#### A.9. Comparison of the efficiency of GTNs and FastGTNs with GNN based baselines

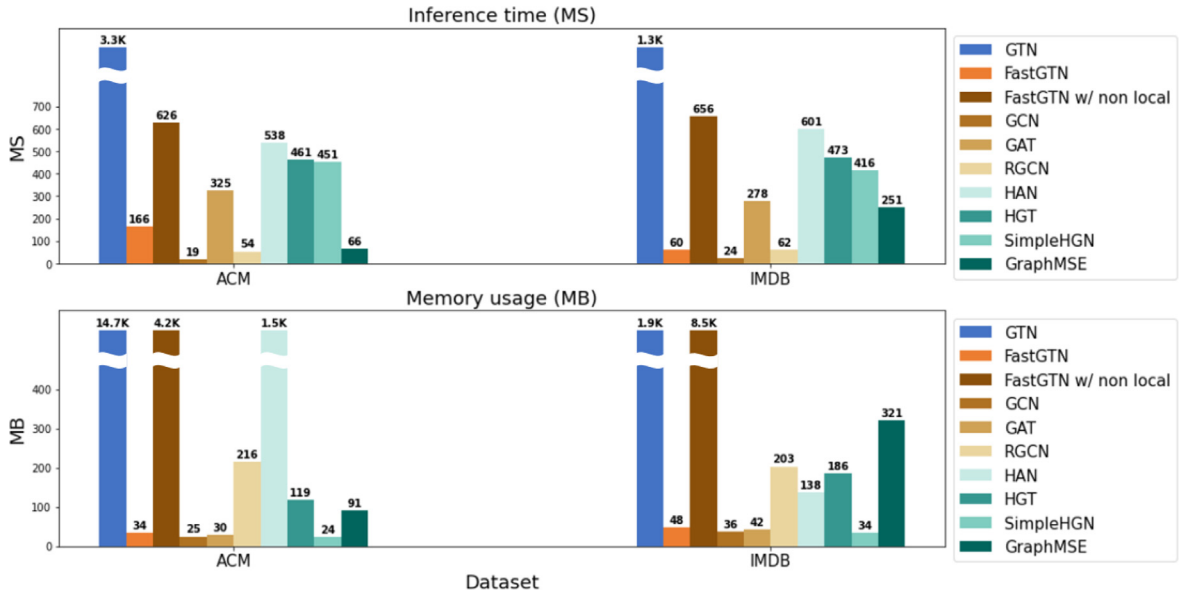
We compared the efficiency of GTNs, FastGTNs, and FastGTNs w/ non-local operations with other GNN based baselines in six datasets that can perform full batch inference of GTNs with a GPU. For a fair comparison, we implemented all baselines in the same library (PyTorch and PyTorch Geometric (Paszke et al., 2019)). As shown in Figs. 11 and 12, FastGTN shows similar inference time (up to 20 ms slow) and memory usage (up to 20MB more) to homogeneous baselines except for GAT. GAT rather takes more inference time (up to 580 ms slow) due to the feature-based attention mechanism. In heterogeneous graphs, as the number of edge types increases, FastGTNs take more inference time (up to 110 ms slow) than GCN and RGCN, but it is up to 600 ms faster than GAT-based heterogeneous baselines (HAN, HGT, SimpleHGN). However, FastGTN w/ non-local operations is relatively slower and uses more memory than other baselines except for GTN. This is because non-local operations calculate semantic proximity between all nodes to select  $K$  nodes with high semantic proximity for each node. We leave improving the efficiency of non-local operations as a future direction.



**Fig. 10.** We visualized performance (F1 score) of FastGTNs according to total layers ( $\# \text{FastGTN layers } (L) \times \# \text{FastGT layers } (K)$ ) on Cora (left) and IMDB (right) datasets. performance is always the best when  $L$  is 1 in Cora dataset whereas  $L$  is 2 in IMDB dataset.



**Fig. 11.** Comparisons of inference time (up) and memory usage (down) between GTNs, FastGTNs and other baselines on homogeneous graph datasets (x-axis).



**Fig. 12.** Comparisons of inference time (up) and memory usage (down) between GTNs, FastGTNs and other baselines on heterogeneous graph datasets (x-axis).

## References

- Abu-El-Haija, S., Perozzi, B., Kapoor, A., Alipourfard, N., Lerman, K., Harutyunyan, H., et al. (2019). Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing. In *International conference on machine learning*.
- Bruna, J., Zaremba, W., Szlam, A., & Lecun, Y. (2014). Spectral networks and locally connected networks on graphs. In *International conference on learning representations*.
- Chen, M., Wei, Z., Huang, Z., Ding, B., & Li, Y. (2020). Simple and deep graph convolutional networks. In *International conference on machine learning*.
- Dai, H., Nazi, A., Li, Y., Dai, B., & Schuurmans, D. (2020). Scalable deep generative modeling for sparse graphs. In *International conference on machine learning*.
- Defferrard, M., Bresson, X., & Vandergheynst, P. (2016). Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in neural information processing systems*.
- Franceschi, L., Niepert, M., Pontil, M., & He, X. (2019). Learning discrete structures for graph neural networks. In *International conference on machine learning*.
- Gao, H., & Ji, S. (2019). Graph u-nets. In *International conference on machine learning*.
- Grover, A., & Leskovec, J. (2016). Node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD International conference on knowledge discovery and data mining*.
- Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. In *Advances in neural information processing systems*.
- Hu, Z., Dong, Y., Wang, K., & Sun, Y. (2020). Heterogeneous graph transformer. In *Proceedings of the web conference 2020*.
- Huang, X., Li, J., & Hu, X. (2017). Label informed attributed network embedding. In *Proceedings of the tenth ACM International conference on web search and data mining*.
- Jin, W., Ma, Y., Liu, X., Tang, X., Wang, S., & Tang, J. (2020). Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD International conference on knowledge discovery & data mining*.
- Kim, R., So, C. H., Jeong, M., Lee, S., Kim, J., & Kang, J. (2019). HATS: A hierarchical graph attention network for stock movement prediction. arXiv:1908.07999.
- Kipf, T. N., & Welling, M. (2016a). Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907.
- Kipf, T. N., & Welling, M. (2016b). Variational graph auto-encoders. In *NIPS Workshop on bayesian deep learning*.
- Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International conference on learning representations*.
- Li, Q., Han, Z., & Ming Wu, X. (2018). Deeper insights into graph convolutional networks for semi-supervised learning.
- Li, Y., Jin, Y., Song, G., Zhu, Z., Shi, C., & Wang, Y. (2021). GraphMSE: Efficient meta-path selection in semantically aligned feature space for graph neural networks. In *Proceedings of the AAAI Conference on artificial intelligence*.
- Li, Y., Vinyals, O., Dyer, C., Pascanu, R., & Battaglia, P. (2018). Learning deep generative models of graphs. In *International conference on machine learning*.
- Liao, R., Li, Y., Song, Y., Wang, S., Hamilton, W., Duvenaud, D. K., et al. (2019). Efficient graph generation with graph recurrent attention networks. In *Advances in neural information processing systems*.
- Linmei, H., Yang, T., Shi, C., Ji, H., & Li, X. (2019). Heterogeneous graph attention networks for semi-supervised short text classification. In *Proceedings of the 2019 Conference on empirical methods in natural language processing*.
- Liu, Z., Chen, C., Li, L., Zhou, J., Li, X., Song, L., et al. (2019). Geniepath: Graph neural networks with adaptive receptive paths. In *Proceedings of the AAAI conference on artificial intelligence*.
- Liu, M., Gao, H., & Ji, S. (2020). Towards deeper graph neural networks. In *Proceedings of the 26th ACM SIGKDD International conference on knowledge discovery & data mining*.
- Lv, Q., Ding, M., Liu, Q., Chen, Y., Feng, W., He, S., et al. (2021). Are we really making much progress? Revisiting, benchmarking and refining heterogeneous graph neural networks. In *Proceedings of the 27th ACM SIGKDD Conference on knowledge discovery & data mining*.
- Ma, Y., Wang, S., Aggarwal, C. C., & Tang, J. (2019). Graph convolutional networks with eigenpooling. In *Proceedings of the 25th ACM SIGKDD International conference on knowledge discovery & data mining*.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems*.
- Schlichtkrull, M., Kipf, T. N., Bloem, P., Van Den Berg, R., Titov, I., & Welling, M. (2018). Modeling relational data with graph convolutional networks. In *European semantic web conference*. Springer.
- Shi, C., Li, Y., Zhang, J., Sun, Y., & Philip, S. Y. (2016). A survey of heterogeneous information network analysis. *IEEE Transactions on Knowledge and Data Engineering*.
- Tang, J., Qu, M., Wang, M., Zhang, M., Yan, J., & Mei, Q. (2015). LINE: Large-scale information network embedding. In *WWW*.
- Teru, K. K., Denis, E., & Hamilton, W. L. (2020). Inductive relation prediction by subgraph reasoning. In *Proceedings of the international conference on machine learning*.
- Vashishth, S., Sanyal, S., Nitin, V., & Talukdar, P. (2020). Composition-based multi-relational graph convolutional networks. In *International conference on learning representations*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., et al. (2017). Attention is all you need. In *International conference on machine learning*.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2018). Graph attention networks. In *International conference on learning representations*.
- Wang, X., He, X., Cao, Y., Liu, M., & Chua, T.-S. (2019). Kgat: Knowledge graph attention network for recommendation. In *Proceedings of the 25th ACM SIGKDD International conference on knowledge discovery & data mining*.
- Wang, X., Ji, H., Shi, C., Wang, B., Ye, Y., Cui, P., et al. (2019). Heterogeneous graph attention network. In *The world wide web conference*.
- Wu, J., He, J., & Xu, J. (2019). Net: Degree-specific graph neural networks for node and graph classification. In *Proceedings of the 25th ACM SIGKDD International conference on knowledge discovery & data mining*.
- Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2019). How powerful are graph neural networks? In *International conference on learning representations*.
- Xu, K., Li, C., Tian, Y., Sonobe, T., Kawarabayashi, K.-i., & Jegelka, S. (2018). Representation learning on graphs with jumping knowledge networks. In *International conference on machine learning*.
- Ying, Z., You, J., Morris, C., Ren, X., Hamilton, W., & Leskovec, J. (2018). Hierarchical graph representation learning with differentiable pooling. In *Advances in neural information processing systems*.
- You, J., Liu, B., Ying, Z., Pande, V., & Leskovec, J. (2018). Graph convolutional policy network for goal-directed molecular graph generation. In *Advances in neural information processing systems*.
- You, J., Ying, R., Ren, X., Hamilton, W. L., & Leskovec, J. (2018). Graphrnn: Generating realistic graphs with deep auto-regressive models. In *International conference on machine learning*.
- Yun, S., Jeong, M., Kim, R., Kang, J., & Kim, H. J. (2019). Graph transformer networks. 32, (pp. 11983–11993).
- Zhang, M., & Chen, Y. (2018). Link prediction based on graph neural networks. In *Advances in neural information processing systems*.
- Zhang, C., Song, D., Huang, C., Swami, A., & Chawla, N. V. (2019). Heterogeneous graph neural network. In *Proceedings of the 25th ACM SIGKDD International conference on knowledge discovery & data mining*.
- Zhang, Y., Xiong, Y., Kong, X., Li, S., Mi, J., & Zhu, Y. (2018). Deep collective classification in heterogeneous information networks. In *Proceedings of the 2018 World wide web conference on world wide web*.